

# Foundational Constraint Solving for Expressive Refinement Typing

ANONYMOUS AUTHOR(S)

SMT-based program verifiers are hamstrung by two problems: *expressiveness*, because predictable verification restricts to the boundaries of SMT decidability, and *trust*, because the solver is a large, unverified artifact whose soundness bugs may quietly compromise every tool built on it. We present FLEX, a foundational Constrained Horn Clause (CHC) solver implemented in LEAN, that reduces the trusted base to the kernel alone, and allows using LEAN’s entire proof ecosystem to verify low-level systems code, via three contributions. First, FLEX encodes CHCs as plain LEAN propositions where the Horn variables are existentially bound predicates, and shows how to implement CHC solvers as tactics (meta-programs) that compute kernel-checkable proofs of the CHC propositions. Second, we show how to implement two verified CHC generators in LEAN: a Floyd-Hoare style generator for an imperative language, and a refinement-type-based generator for a functional calculus, which can be composed with the solving tactics to yield the first end-to-end foundational CHC-based verifiers. Finally, we show how FLEX allows us to leapfrog the expressiveness limitations of SMT by unleashing LEAN’s entire ecosystem of proof machinery to prove arbitrary functional correctness properties of various low-level Rust libraries using the FLUX refinement type checker, and demonstrate the viability of FLEX as a trustworthy CHC backend, by showing it automatically discharges 95.7% of the CHCs from FLUX’s benchmark suite.

## 1 Introduction

SMT solvers are the beating heart of automated verification tools [2, 22, 44, 45, 48, 67, 73] that convert messy, impure code into purely logical formulas whose validity, automatically checked by the solver, guarantees the code meets critical correctness and security requirements [31, 43, 46, 61, 79].

However, experience shows that SMT solvers are hamstrung by two problems: *expressiveness* and *trust*. First, the solvers implement decision procedures for a fixed set of decidable theories [55], and use heuristics to reason about formulas that lie beyond [17]. While the decision procedures make short work of formulas that arise in proving specifications over integers, arithmetic, or uninterpreted functions, verification becomes a slog for more *expressive* specifications with quantifiers or user-defined predicates, or even when reasoning with decidable but hard theories like bit-vectors. Here, the incomplete heuristics make solving unpredictable due to “proof instability” [49, 78], and the black-box nature of SMT — where it simply says whether a formula is valid or not — leaves the programmer throwing darts in the dark to divine what extra hints the solver might require to complete a proof [26, 53]. Second, SMT engines are substantial artifacts: Z3 and CVC4 clock in at around half a million lines of C++, with known soundness issues [11, 74–76] — a significant *trusted computing base* even before accounting for the verification formula generators.

**FLEX** We present FLEX, a new approach to engineering program verifiers using foundational Horn constraint solving. FLEX is based on the observation that the task of verification can be split into a front-end that generates *Constrained Horn Clauses* (CHCs) and a back-end that determines their satisfiability [6]. A CHC is a logical formula over a set of *Horn variables* representing unknown program invariants, which must satisfy requirements captured by conjunctions and implications over those variables. Our key insight is that CHCs are *existentially quantified propositions* in a foundational logic, such as that of LEAN or ROCQ or ISABELLE. Hence, we can entirely leapfrog the expressiveness and trust limitations of SMT by implementing a CHC solver *within* a foundational prover, as a meta-program that *computes proofs of* CHC propositions. In this paper, we develop, implement and evaluate this approach via three contributions.

**1. Foundational Solving (§4)** Our first contribution is to show how CHCs can be shallowly embedded as LEAN propositions, and then show how existing (SMT-based) CHC solving techniques can be reimagined as tactics that *reduce* away the Horn variables from the proposition, leaving behind a plain formula that can be discharged with existing LEAN machinery. To solve CHCs our

tactics must compute *witness predicates* for the existentially bound Horn variables. Following [14], we observe that these variables can be partitioned into *acyclic* variables which can be solved exactly or *cyclic* variables which must be approximated because they arise respectively from “straight-line” or “looping” code. We show how to compute this partition, and then how to adapt the Fusion [14] and Predicate Abstraction [21, 62] CHC solving algorithms to the foundational setting, where we can parameterize them with extensible *oracles* that enable synthesizing invariants over user-defined theories. Of course, a foundational kernel will not take any reduction of the Horn variables on faith: it demands proof that the reduction is legitimate. A key contribution of our tactics is to show how to exploit the structure of CHCs and their solutions to *automatically construct certificates* that the reduced CHC logically implies the original one, yielding the first foundational CHC solver.

**2. Foundational Generation (§5)** Our second contribution is to implement two foundational CHC *generators*, which, when connected with FLEX, yield the first end-to-end CHC-based verifiers. As a warmup, we develop a *Floyd-Hoare* style generator [24, 35] for an imperative language IMP, that follows the textbook recipe [56, 60] but crucially uses Horn variables to defer loop-invariant synthesis to the back-end solver. Next, we present the first mechanization of algorithmic *refinement typing* [38], by deeply embedding a core functional calculus  $\lambda_{RK}$  in LEAN, formalizing a big-step operational semantics for it, defining a semantic interpretation for refinement types, implementing a CHC generator that uses Horn variables for unknown *refinements*, and proving that if the generated CHCs are satisfiable, then the source yields — without getting stuck — a value in its type’s interpretation. Together these verifiers offer a blueprint for how to engineer CHC-based verification tools entirely within a proof assistant.

**3. Expressive Verification (§6)** Our final contribution demonstrates FLEX’s ultimate payoff: liberating verification from SMT’s expressiveness and trust limitations by unleashing LEAN’s entire ecosystem of proof machinery — theories, tactics, agents, and all — to prove arbitrary functional correctness properties of low-level Rust code. The FLUX verifier uses refinement types to distill impure Rust code — with the classic challenges of mutable state like references and loops, extended with hairy modern constructs like traits, closures and asynchrony — into purely logical CHCs. Previously, the expressiveness of FLUX’s specifications was carefully restricted to ensure the CHCs fell within the boundaries of SMT decidability. FLEX tears down this wall, by using LEAN as a backend to discharge functional correctness specifications, as illustrated by a variety of case studies: correctness of in-place sorting algorithms; correctness of a hash table with chaining for collision resolution; modular arithmetic and bit-vector properties required for secure memory isolation in the Tock embedded kernel [61]; and proving that a ring-buffer-backed dequeue from the embedded kernel [64] never reads from uninitialized memory. Further, we show that FLEX is viable as a *trustworthy* back-end CHC solver that is able to automatically discharge more than 95% of the 880 CHCs that arise in FLUX’s existing benchmark suite spanning 20K lines of Rust source. This high degree of automation is made possible by FLEX’s certificate generation machinery which successfully reduces *every* acyclic variable across 369 CHCs without search, compared to a baseline of LEAN’s general-purpose grind and aesop which close only 67% and 90% of CHCs while taking 18 to 34× longer to complete the proof.

The technical supplement includes FLEX implementation, verifiers (IMP and  $\lambda_{RK}$ ) mechanization, case-studies (§ 6.1), and paper proofs for (§4).

## 2 Overview

Let us begin with an overview of FLEX. We start by recalling how refinement type checking reduces to *generating* CHC constraints (§ 2.1). Next, we illustrate our first contribution by showing how FLEX *solves* the constraints in a foundational setting (§ 2.2). Then, we demonstrate our second

|     |                                                                   |                  |                                                                         |
|-----|-------------------------------------------------------------------|------------------|-------------------------------------------------------------------------|
| 99  | <code>fn foreach&lt;F&gt;(lo:usize, hi:usize, mut f:F)</code>     | $c_{for} \doteq$ | $\exists \kappa: Int \rightarrow Int \rightarrow Int \rightarrow Prop.$ |
| 100 | <code>where F: FnMut(usize{v:lo&lt;=v &amp;&amp; v&lt;hi})</code> |                  | $\forall lo\ hi: Int.$                                                  |
| 101 | <code>{</code>                                                    |                  | $\kappa(lo, hi, lo)$                                                    |
| 102 | <code>let mut i = lo;</code>                                      |                  | $\wedge \forall i: Int.$                                                |
| 103 | <code>while i &lt; hi {</code>                                    |                  | $\kappa(i, hi, lo) \rightarrow i < hi \rightarrow$                      |
| 104 | <code>  f(i);</code>                                              |                  | $lo \leq i \wedge i < hi$                                               |
| 105 | <code>  i += 1;</code>                                            |                  | $\wedge \kappa(i + 1, hi, lo)$                                          |
| 106 | <code>}</code>                                                    |                  |                                                                         |
| 107 | <code>}</code>                                                    |                  |                                                                         |
| 108 | <code>fn dot(n:usize, x:&amp;[f64][n], y:&amp;[f64][n])</code>    | $c_{dot} \doteq$ | $\exists \kappa: Int \rightarrow Int \rightarrow Prop.$                 |
| 109 | <code>-&gt; f64</code>                                            |                  | $\forall n: Int.$                                                       |
| 110 | <code>{</code>                                                    |                  | $\forall j: Int. \kappa(j, n) \rightarrow$                              |
| 111 | <code>let mut res = 0.0;</code>                                   |                  | $j < n$                                                                 |
| 112 | <code>let body =  j  { res += x[j] * y[j] };</code>               |                  | $\wedge \forall v: Int. (0 \leq v \wedge v < n) \rightarrow$            |
| 113 | <code>foreach(0, n, body);</code>                                 |                  | $\kappa(v, n)$                                                          |
| 114 | <code>res</code>                                                  |                  |                                                                         |
| 115 | <code>}</code>                                                    |                  |                                                                         |

Fig. 1. (L) Rust with FLUX specifications, (R) verification CHCs generated by FLUX

contribution where we use FLEX’s solvers to implement foundational verifiers in LEAN (§ 2.3). Finally, we conclude by showing how by tapping into LEAN’s full machinery, FLEX lets FLUX verify arbitrarily expressive specifications of Rust code (§ 2.4).

## 2.1 Horn Constraints

Consider the two Rust functions shown on the left in Fig. 1. The function `foreach` is a higher-order loop, that takes as input three parameters: a *range* denoted by a lower bound `lo` and upper bound `hi`, and a body `f`. The function uses a `while`-loop to evaluate `f(i)` for each index `i` in the supplied range. The function `dot` takes as input two Rust slices `x` and `y` of dimension `n`, and then computes the dot product of the two slices by invoking `foreach` on the range `[0, n)`, and the *closure* `body` that accumulates the contribution of the  $j^{th}$  dimension of each slice.

**Specification: Refinement Types** Suppose we wish to statically verify that `dot` does not *panic* by performing an out-of-bounds access. To do so, the programmer writes FLUX *refinement type specifications* for each function. The specification for `foreach` says that the closure `f` will only be called at indices `v` that are in the range `[lo, hi)`, as stipulated by the closure’s precondition: an existential refinement on the input type of `f`. For `dot`, the specification requires callers pass in slices `x` and `y` with `n` elements, as prescribed by the indexed slice type `&[f64][n]`.

**Verification: Constraints** To *verify* both functions in Fig. 1, a refinement type verifier like FLUX runs the type checker to generate a *verification constraint* shown on the function’s right. Each constraint is a *Constrained Horn Clause* (CHC) that comprises the following elements. At the top, a CHC has some number of existentially quantified *Horn variables* (denoted by  $\kappa$ ) which represent *unknown* refinements or invariants. Next, the constraint itself, comprises a sequence of nested (1) *universal* bindings, corresponding to statically unknown program values; (2) *conjunctions*, corresponding to multiple requirements; (3) *implications*, corresponding to hypotheses about program values; ending in a (4) *head* obligation that must hold under the preceding hypotheses.

**CHC for `foreach`** The constraint  $c_{for}$  has one Horn variable  $\kappa : Int \rightarrow Int \rightarrow Int \rightarrow Prop$  representing the `while`-loop invariant over the counter `i`, upper bound `hi`, and lower bound `lo`. The first

$$\begin{array}{ccc}
\exists \kappa : \text{Int} \rightarrow \text{Int} \rightarrow \text{Prop}. & & \\
\forall n : \text{Int}. & & \forall n : \text{Int}. \\
\forall j : \text{Int}. \kappa(j, n) \rightarrow & \xrightarrow{\text{Zap}} & \forall j : \text{Int}. \exists v. 0 \leq v \wedge v < n \wedge j = v \rightarrow \xrightarrow{\text{grind}} \text{No} \\
j < n & & j < n & & \text{goals.} \\
\wedge \forall v : \text{Int}. (0 \leq v \wedge v < n) \rightarrow & & \wedge \forall v : \text{Int}. (0 \leq v \wedge v < n) \rightarrow & & \\
\kappa(v, n) & & \top & & 
\end{array}$$

Fig. 2. (L) The CHC for `dot`, (C) Zap the *acyclic*  $\kappa$  replacing head occurrences with  $\top$  and body occurrences with the strongest solution, (R) grind then discharges the residual VC to verify `dot`.

(head) conjunct  $\kappa(lo, hi, lo)$  says the invariant must hold at loop entry (when  $i = lo$ ). The second conjunct is the inductive step: if the invariant holds at  $i$  and the loop guard  $i < hi$  is true, then we have two head obligations. (a) First, the closure’s *precondition*  $lo \leq i \wedge i < hi$  must hold, ensuring  $f$  is called within the specified range. (b) Second, the invariant must be preserved at  $i + 1$ , as mandated by the head  $\kappa(i + 1, hi, lo)$ .

**CHC for `dot`** Closures would be quite tiresome to use if programmers had to explicitly annotate them with their contracts. To relieve them of this tedium, FLUX introduces a Horn variable  $\kappa : \text{Int} \rightarrow \text{Int} \rightarrow \text{Prop}$  to represent the unknown input refinement for the closure *body*. The first conjunct of the constraint says that whenever the closure executes with an index  $j$ , which is assumed to satisfy the closure’s input refinement  $\kappa(j, n)$ , the bound  $j < n$  needed to safely access  $x[j]$  and  $y[j]$  must hold. The second conjunct is obtained from the *subtyping* obligation between the function type of the argument (*body*) and of the `foreach`’s parameter ( $f$ ). Specifically, by contravariant input subtyping, the constraint says that since `foreach` calls the closure with  $v$  where  $0 \leq v \wedge v < n$  (as it is called with  $lo = 0$ ,  $hi = n$ ), all such values must satisfy the input refinement  $\kappa(v, n)$ .

## 2.2 Foundational Solvers

The verification constraints (CHCs) distill Rust’s stateful, temporal, and impure semantics into purely logical propositions — precisely the kind of thing that proof assistants were designed to prove. Except for one major challenge: proofs of CHC propositions must supply *explicit witnesses* for the existentially bound Horn variables, satisfying the head obligations on those variables. (Indeed, we tried proving some CHCs by hand — and with coding agents — but it was readily apparent that this direct route was utterly impractical.)

FLEX addresses this challenge by reimagining two existing CHC-solving algorithms as *rewriting tactics* for the foundational setting.

**Cyclic vs Acyclic Variables** The Horn variables  $\kappa$  in CHCs come in two flavors. The variable  $\kappa$  may be *cyclic*, which means, roughly (we defer a precise definition to § 4.1), that it appears in the hypotheses of a clause where it also appears in the head; and otherwise we say it is *acyclic*. For example, in the constraint  $c_{for}$  in Fig. 1,  $\kappa$  is cyclic as it appears in both the hypothesis  $\kappa(i, hi, lo)$  and the head  $\kappa(i + 1, hi, lo)$  of the inductive clause. In contrast, the variable  $\kappa$  in  $c_{dot}$  is acyclic as it appears only in hypotheses in the first clause and only in head in the second.

**Solving Acyclic Variables** For an *acyclic* variable  $\kappa$  we can compute an exact closed form solution, perhaps in terms of *other* variables. The Fusion algorithm of [14] shows how to compute the *strongest* solution, informally by taking the *conjunction* of all the hypotheses that occur along the path to each head occurrence of that  $\kappa$ , and then computing the *disjunction* over all the paths leading to head occurrences of the  $\kappa$ .

In FLEX, the above algorithm is implemented in a tactic Zap detailed in § 4.2, and shown in action in Fig. 2. On the left, we have the original CHC proposition. First, Zap computes the strongest

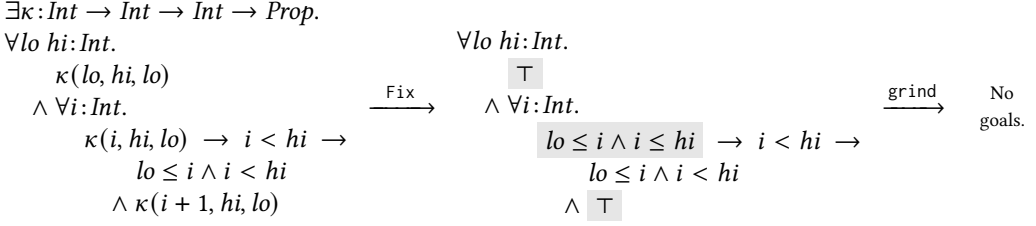


Fig. 3. (L) The CHC for `foreach`, (C) `Fix` uses qualifiers to reduce the *cyclic*  $\kappa$ , replacing head occurrences with  $\top$  and body occurrences with the solution, (R) `grind` then discharges the residual VC to verify `foreach`.

solution by traversing the CHC to the (single) head application of  $\kappa$ , collecting the hypotheses along the path, and existentially quantifying the universally quantified binders:

$$\kappa \doteq \lambda z, n. \exists v. 0 \leq v \wedge v < n \wedge z = v$$

Next, the Zap tactic traverses the CHC, *reducing* all the head applications of  $\kappa$  to  $\top$  (i.e., *True*) as they are guaranteed to hold by construction, and the body applications of  $\kappa$  with the computed solution. These reductions are highlighted in gray. The result is a horn variable free *verification condition* which is discharged by LEAN’s `grind` tactic.

**Solving Cyclic Variables** The Horn variable  $\kappa$  in  $c_{for}$  is cyclic: it appears as a hypothesis along the same path where it is the goal. Hence, we cannot apply the technique described above as it would loop indefinitely! Instead, FLEX has a `Fix` tactic that implements a form of abstract interpretation called *predicate abstraction* [21, 28]. This tactic proceeds in three steps.

*Step 1: Qualifiers* The `Fix` tactic solves cyclic variables as *conjunctions of predicates* from a library of *qualifier templates* [62]. In FLEX, a qualifier is any LEAN predicate tagged with `@[qual]`, e.g.,

```

@[qual] def q_le (a b : Int) := a ≤ b
@[qual] def q_lt (a b : Int) := a < b
@[qual] def q_eq (a b : Int) := a = b
    
```

*Step 2: Initial Assignment* Next, `Fix` uses the qualifiers to compute an initial assignment which instantiates *all* the qualifiers with all the possible  $\kappa$  parameters that yield a well-typed LEAN predicate. For example, an initial assignment for the variable  $\kappa$  from  $c_{for}$  would be

$$\kappa(z, hi, lo) \doteq \{z \leq hi, hi \leq z, z \leq lo, lo \leq z, \dots z < hi, hi < z, z < lo, lo < z, \dots\}$$

*Step 3: Weaken till Fixpoint* Finally, `Fix` enters an iterative weakening loop where we use an *oracle* — which could be `grind` or any other theory specific decision procedure in LEAN — to *discard* candidates that *cannot* be proven to hold at some CHC head. Once we have eliminated the impossible, whatever candidates remain, must be the truth. Hence, `Fix` returns a solution comprising the conjunction of all the candidates that remain standing. For our example, two candidates remain, yielding the solution  $\kappa \doteq \lambda z, hi, lo. lo \leq z \wedge z \leq hi$ . Like Zap, `Fix` uses the computed solution to reduce away  $\kappa$  by replacing head occurrences with  $\top$  and body occurrences with the computed fixpoint, as shown in Fig. 3, yielding a plain VC that can be discharged by `grind`.

**Certifying Reductions** A key challenge that FLEX addresses is that in the foundational setting, we cannot blithely rewrite the constraint with an alleged solution: instead, we must conclusively prove that the reduction is legitimate. Thus, whenever FLEX’s CHC solving tactics Zap and `Fix` reduce a constraint  $c$  to  $c'$  by computing the solution for a single variable  $\kappa$ , they traverse  $c$  and the solution to construct an explicit *certificate* that demonstrates that any proof of the reduced constraint  $c'$  yields a proof of the original  $c$ . To this end, we observe that the structure of constraints, solutions

| Constraint        | Solution                | Certificate                                     |                                                                                                                            |
|-------------------|-------------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| $c_1 \wedge c_2$  | $solc_1 \vee solc_2$    | $\text{inl} / \text{inr}$                       | $\exists \kappa : \text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \rightarrow \text{Prop}$ |
| $\forall x. c$    | $\exists x. solc$       | $\langle x, \cdot \rangle$                      | $\forall n : \text{Int}. (2 \leq n \wedge \text{fib}(n) < \text{MAX} \wedge n < \text{MAX}) \rightarrow$                   |
| $p \rightarrow c$ | $p \wedge solc$         | $\langle h, \cdot \rangle$                      | $\kappa(1, 2, 2, n)$                                                                                                       |
| $\kappa(\bar{t})$ | $\bigwedge_i z_i = t_i$ | $\langle \text{rfl}, \dots, \text{rfl} \rangle$ | $\wedge \forall \text{prev curr } i : \text{Int}. \kappa(\text{prev}, \text{curr}, i, n) \rightarrow$                      |
|                   |                         |                                                 | $i \geq n \rightarrow \text{curr} = \text{fib}(n)$                                                                         |
|                   |                         |                                                 | $\wedge i < n \rightarrow \text{prev} + \text{curr} \leq \text{MAX} \wedge$                                                |
|                   |                         |                                                 | $\kappa(\text{curr}, \text{prev} + \text{curr}, i + 1, n)$                                                                 |

Fig. 4. (L) Triad of Constraints, Solutions and Certificates (§ 4.3); (R) CHC from fib\_loop in Fig. 7.

|                                             |                                                                                                                             |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| @[grind] def fib (n : Int) : Int :=         | $\exists \kappa : \text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \rightarrow \text{Prop}.$ |
| if n ≤ 1 then 1 else fib(n-1) + fib(n-2)    | $\forall n : \text{Int}.$                                                                                                   |
| termination_by n.toNat                      | $n = a \rightarrow 2 \leq a \rightarrow \kappa(n, 1, 2, 2)$                                                                 |
|                                             | $\wedge \forall n p x i : \text{Int}.$                                                                                      |
| @[qual] def q1 (v i : Int) := v = fib i     | $\kappa(n, p, x, i) \rightarrow i < n \rightarrow \kappa(n, x, p + x, i + 1)$                                               |
| @[qual] def q2 (v i : Int) := v = fib (i-1) | $\wedge \forall n p x i : \text{Int}.$                                                                                      |
|                                             | $\kappa(n, p, x, i) \rightarrow n \leq i \rightarrow x = \text{fib}(a)$                                                     |
| theorem fibLoop (a : Int) :                 | $\xrightarrow{\text{fix}}$                                                                                                  |
| = (λs => s "n" = a ∧ a ≥ 2)                 | $\forall n : \text{Int}.$                                                                                                   |
| <  prev := 1; x := 2; i := 2;               | $n = a \rightarrow 2 \leq a \rightarrow \top$                                                                               |
| while i < n do (                            | $\wedge \forall n p x i : \text{Int}.$                                                                                      |
| next := prev + x ;                          | $\dots \rightarrow i < n \rightarrow \top$                                                                                  |
| prev := x ;                                 | $\wedge \forall n p x i : \text{Int}.$                                                                                      |
| x := next ;                                 | $n = a \wedge i \leq n \wedge p = \text{fib}(i-1) \wedge x = \text{fib}(i) \rightarrow$                                     |
| i := i + 1)  >                              | $n \leq i \rightarrow x = \text{fib}(a)$                                                                                    |
| (λs => s "x" = fib a)                       |                                                                                                                             |
| := by generate; fix; grind                  |                                                                                                                             |

Fig. 5. End-to-end Verification for IMP: (L) Specification of fib (top) and imperative implementation (bot), (R) Generated VC (top) and reduced constraint (bot).

and hence, their certificates, form a triad summarized in Fig. 4 (and detailed in § 4.3). Their rhyming structure lets us mirror the traversal used to compute the solution to construct an explicit LEAN proof term (using the matching constructs in Fig. 4) that justifies why it is sound to replace each head  $\kappa$ -application with  $\top$  at that juncture in the constraint. These proofs are then chained together to show that any proof of the  $\kappa$ -free proposition is also a proof of the original CHC, yielding an end-to-end guarantee.

### 2.3 Foundational Verifiers

Our second contribution is to use the solvers to develop the first foundational CHC-based verifiers. First, a Floyd-Hoare style verifier [24, 35] for an imperative language IMP where the Horn variables let us synthesize *loop invariants* [6]. Second, a verifier for a core functional calculus extended with refinement types  $\lambda_{RK}$ , which uses Horn variables to infer unknown *refinements*. In each case, we formalize the source language's operational semantics and implement a CHC generator that is proved sound against the semantics. FLEX can then discharge the generated CHCs, highlighting two key benefits: first that it can be used as a foundational "back-end" verifier for stateful or functional programs in LEAN; second that it can reuse LEAN's proof ecosystem to check specifications over arbitrary user-defined functions.

|     |                                                                                                        |                                                                                                    |
|-----|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| 295 | $\text{max} : x:\text{Int} \rightarrow y:\text{Int} \rightarrow \{v:\text{Int} \mid \kappa(v, y, x)\}$ | $\exists \kappa:\text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \rightarrow \text{Prop}.$ |
| 296 | $\text{max} \doteq \lambda x y \rightarrow$                                                            | $\forall x y:\text{Int}.$                                                                          |
| 297 | $\text{let } c = x \leq y \text{ in}$                                                                  | $x \leq y \rightarrow \kappa(y, y, x)$                                                             |
| 298 | $\text{if } c \text{ then } y \text{ else } x$                                                         | $\wedge \neg(x \leq y) \rightarrow \kappa(x, y, x)$                                                |
| 299 |                                                                                                        | $\wedge \forall v:\text{Int}.$                                                                     |
| 300 | $\text{ex} \doteq \text{let } a = 6 \text{ in}$                                                        | $\kappa(v, 7, 6) \rightarrow v = 7$                                                                |
| 301 | $\text{let } b = 7 \text{ in}$                                                                         |                                                                                                    |
| 302 | $\text{max } a \ b$                                                                                    | $\xrightarrow{\text{zap}}$                                                                         |
| 303 |                                                                                                        | $\forall x y:\text{Int}.$                                                                          |
| 304 | <b>example:</b> $\exists K, \emptyset \vdash_K \text{ex} \Leftarrow \{v:\text{Int} \mid v = 7\}$       | $x \leq y \rightarrow \top$                                                                        |
| 305 | := <b>by</b>                                                                                           | $\wedge \neg(x \leq y) \rightarrow \top$                                                           |
| 306 | generate                                                                                               | $\wedge \forall v:\text{Int}.$                                                                     |
| 307 | zap                                                                                                    | $(6 \leq 7 \wedge v = 7) \vee (\neg 6 \leq 7 \wedge v = 6) \rightarrow v = 7$                      |
| 308 | grind                                                                                                  |                                                                                                    |

Fig. 6. End-to-end Verification for  $\lambda_{RK}$ : (L) Typing an  $\lambda_{RK}$  that computes the larger of two inputs, (R) CHC generated by typing.

**Loop Invariant Synthesis for IMP** Fig. 5 shows our verifier for IMP in action. On the left, we first have a *specification* for `fib` as a plain LEAN function, and below, a textbook imperative loop that computes the  $n^{\text{th}}$  Fibonacci number. We use FLEX to prove the (partial) Floyd-Hoare triple that says that when the loop is executed from a state where the value of the variable `n` equals `a` with  $a \geq 2$  then (if it exits) the value of `x` equals `fib(a)`.

The proof of the triple is in two steps. First, the `generate` tactic reduces the assertion to the CHC shown on the top right. This CHC has a single cyclic Horn variable  $\kappa$  that represents the unknown loop invariant. The classical weakest-precondition based VC generation [18] yields a CHC with constraints that check (1) the invariant holds upon loop entry; (2) the invariant is inductive (re-established by the body) and (3) the specified postcondition holds on exit. Second, FLEX’s `Fix` tactic reduces the CHC using predicate abstraction to synthesize a solution for the  $\kappa$  using the qualifiers shown above, yielding the constraint shown below, discharged by `grind`.

**Refinement Synthesis for  $\lambda_{RK}$**  Fig. 6 shows our verifier for  $\lambda_{RK}$  in action. On the left, we have a function `max` that returns the larger of its two inputs with a return type carrying an unknown refinement  $\kappa(v, y, x)$ . The expression `ex` applies `max` to 6 and 7. We use FLEX to prove that `ex` can be typed as  $\{v:\text{Int} \mid v = 7\}$  under some typing environment  $K$  that assigns  $\kappa$  a concrete predicate.

The proof proceeds in two steps. First, the `generate` tactic runs  $\lambda_{RK}$ ’s bidirectional type checker to emit the CHC shown on the top right. This CHC has a single *acyclic* Horn variable  $\kappa:\text{Int} \rightarrow \text{Int} \rightarrow \text{Int} \rightarrow \text{Prop}$  representing the unknown return refinement. Type checking the body of `max` yields two head constraints: if  $x \leq y$  then the return value `y` must satisfy  $\kappa(y, y, x)$ , and symmetrically  $\kappa(x, y, x)$  in the else branch. The call site `max 6 7` checked against  $\{v:\text{Int} \mid v = 7\}$  contributes the final clause: any  $v$  satisfying  $\kappa(v, 7, 6)$  must equal 7. Second, FLEX’s `Zap` tactic reduces the CHC: since  $\kappa$  is acyclic, it computes its strongest solution, replaces each head occurrence with  $\top$  and each body occurrence with the computed solution (highlighted in gray on the bottom right), yielding a plain VC that `grind` dispatches.

## 2.4 Expressive Refinement Typing

Our last contribution shows how embedding CHCs into LEAN liberates refinement types from the limitations of SMT-solver theories, enabling their use for proving functional correctness properties of Rust using the FLUX refinement type checker (§6).

```

344 fn fib_loop(n: usize) -> usize[fib(n)]
345   requires 2 <= n && fib(n) < usize::MAX
346 {
347   let mut prev = 1;
348   let mut curr = 2;
349   let mut i = 2;
350   while i < n {
351     let next = prev + curr;
352     prev = curr;
353     curr = next;
354     i += 1;
355   }
356   return curr;
357 }

```

```

theorem fib_mono: ∀ (i j : Int),
  0 ≤ i → i ≤ j → fib i ≤ fib j
:= by
  intros i j hi hj
  induction j using fib_spec_fib.induct
  generalizing i with grind

def FibLoop_proof : FibLoopCHC := by
  unfold FibLoopCHC
  fix
  have _ : fib (i + 1) <= fib n := by
    apply fib_mono <;> grind
  grind

```

Fig. 7. Expressive Refinement Typing in FLUX

**Specifying Non-Overflow** The left of Fig. 7 shows a Rust function that implements the same imperative fibonacci computation from Fig. 5. However, in the Rust setting, all the variables are fixed-sized (unsigned) integers `usize` and hence, FLUX additionally requires us to prove that various arithmetic operations, notably `prev + curr`, do not overflow. The operation *will* overflow unless the `n` is sufficiently small, as stated by the additional precondition. Thus, when run on this code, FLUX generates a CHC `FibLoopCHC` shown on the right in Fig. 4, that is similar to that in Fig. 5 but with an additional *assumption* that `fib(n) < MAX` and a *head* that checks `prev + curr ≤ MAX`.

**Verifying Non-Overflow** The right side of Fig. 7 shows how we can verify the code by proving `FibLoopCHC` in LEAN: we need simply unfold and use the FLEX tactic `fix`, which uses the qualifiers in Fig. 5 to solve for and reduce the  $\kappa$ , leaving behind the single goal

$$2 \leq i < n, \text{fib}(n) < \text{MAX}, \dots \vdash \text{fib}(i-1) + \text{fib}(i) \leq \text{MAX}$$

that we can discharge by proving and applying a lemma that `fib` is monotonic (`fib_mono`) in LEAN. In the SMT based setting [44, 48] verification failures are opaque. The programmer would have to stare hard at the code to understand the root cause, then prove the monotonicity lemma using the spare (relative to LEAN) affordances of the verifier, and finally instantiate the lemma by cluttering the source with assertions. In contrast, LEAN’s interactive context makes it easy to spot what is *missing*, and its ecosystem of theories and tactics makes it easy to then close the gap.

### 3 Constraints and Proofs

FLEX represents a system of *Constrained Horn Clauses* (CHCs) in a restricted fragment of LEAN terms of type *Prop*. The syntactic structure preserves the information needed for Horn solving, while ensuring that key logical connectives like  $\forall$ ,  $\wedge$ ,  $\rightarrow$  are native to LEAN, thereby allowing us to implement the solver as an ordinary metaprogram over LEAN’s datatype *Expr*.

This representation provides two key benefits. First, it gives us *theories for free* as a LEAN proposition can use formulas over arbitrary LEAN theories to write specifications, allowing us to use arbitrary LEAN definitions to synthesize invariants and use LEAN proof automation — e.g. `omega`, `bv_decide`, `decide`, `grind` or lemmas [70] — to simplify verification, and, we do not need to maintain a deeply embedded AST for each theory, re-using *Expr* allows for inheriting theories of LEAN, keeping the solver simple, but expressive. Second, it provides *soundness for free*, by letting

| <i>Lean Term</i>         | $t, \varphi \in \mathcal{T}$ | <i>Variables</i>                                                                                                                                                                                                                              | $x, h \in \mathcal{V}$ | $\kappa$ -variable                                   | $\kappa \in \mathcal{K} \subset \mathcal{V}$ |
|--------------------------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|------------------------------------------------------|----------------------------------------------|
| <b>Sort</b>              | $b ::=$                      | $Prop \mid Int \mid Nat \mid Bool \mid BitVec\ n \mid \dots$                                                                                                                                                                                  |                        |                                                      |                                              |
| <b>Atoms</b>             | $p ::=$                      | $\varphi$<br>$\mid \kappa(\bar{t})$                                                                                                                                                                                                           |                        | $\kappa$ -free atom<br>horn application              |                                              |
| <b>Constraints</b>       | $c ::=$                      | $p$<br>$\mid c \wedge c$<br>$\mid p \rightarrow c$<br>$\mid \forall x:b. c$                                                                                                                                                                   |                        | head<br>conjunction<br>implication<br>quantification |                                              |
| <b>Closed Constraint</b> | $C ::=$                      | $\exists \bar{\kappa}. c$                                                                                                                                                                                                                     |                        |                                                      |                                              |
| <b>Proof term</b>        | $q ::=$                      | $t \mid \lambda x. q \mid q\ q \mid \langle q, q \rangle \mid q.1 \mid q.2 \mid \text{inl}\ q \mid \text{inr}\ q$<br>$\mid \langle t, q \rangle \mid \text{let } \langle x, h \rangle = q \text{ in } q \mid \text{rfl} \mid \langle \rangle$ |                        |                                                      |                                              |

Fig. 8. Grammar of the CHC fragment of Lean 4 Expr at sort *Prop*, together with the proof-term fragment emitted by our solvers.

us implement CHC solvers (§4) that emit proof terms which can then be checked by the kernel, guaranteeing the soundness of verification without trusting any code beyond the kernel itself.

As a running example throughout this section, consider a constraint that asserts  $x \geq 0$  and an unknown refinement  $\kappa_1$  relating  $x$  to its successor  $v$ :

$$c_0 \doteq \exists \kappa_1. \forall x. 0 \leq x \rightarrow (\forall v. v = x + 1 \rightarrow \kappa_1(v, x)) \wedge (\forall a. \kappa_1(a, x) \rightarrow 1 \leq a) \quad (1)$$

We will point back to  $c_0$  as we introduce each piece of notation below.

### 3.1 Syntax and Semantics

Fig. 8 shows the syntax for CHCs, which follows a nested structure that preserves scoping information our solvers later use to recover the context of each Horn application (§4.2).

**Sorts and Terms** We assume a base family of *sorts*  $b$  which range over arbitrary LEAN types like integers, bit-vectors, sets, maps *etc.* We write  $x_1, x_2, x_3, \dots$  for *variables* of sort  $b$ , and write  $t_1, t_2, t_3, \dots$  for arbitrary LEAN terms of the appropriate sort. In  $c_0$  above,  $x, v$ , and  $a$  are variables of sort *Int*, while  $x + 1$  is a term of sort *Int* and  $1 \leq a$  is a term of sort *Prop*. By convention, when a term is of sort *Prop*, we use the metavariable  $\varphi$  to denote it. Similarly, we use  $h$  to range over variables naming hypotheses, *i.e.*, of sort *Prop*. We assume terms include the trivial propositions  $\top$  and  $\perp$  (of sort *Prop*), which we will use to simplify constraints during solving (§4).

**Horn Variables** A *Horn (refinement) variable* is a LEAN variable  $\kappa : b_1 \rightarrow \dots \rightarrow b_{n_\kappa} \rightarrow Prop$ , and occurs in a constraint as horn applications  $\kappa(\bar{t})$ . We assume variables  $\kappa$  do not appear inside arbitrary terms  $t$ . We will introduce such variables to represent *unknown* refinements, by existentially quantifying them in the constraints. For any constraint  $c$ , we write  $\text{KVars}(c)$  for the set of Horn variables occurring in  $c$ . In  $c_0$ ,  $\kappa_1 : Int \rightarrow Int \rightarrow Prop$  is a horn variable relating two integers.

**Atoms** An *atom*  $p$  is either a  $\kappa$ -free term  $t$  of sort *Prop*, or a *horn application*  $\kappa(\bar{t})$ . In  $c_0$ ,  $0 \leq x$ ,  $v = x + 1$ , and  $1 \leq a$  are  $\kappa$ -free terms, while  $\kappa_1(v, x)$  and  $\kappa_1(a, x)$  are horn applications.

**Constraints** A *Horn constraint*  $c$  is either (1) a *head atom*  $p$ , (2) a *conjunction*  $c_1 \wedge c_2$ , (3) an *implication*  $p \rightarrow c$ , or (4) a *quantification*  $\forall x:b. c$  over a variable  $x$  of sort  $b$ . A *closed constraint*  $C$  is a LEAN proposition of the form  $\exists \kappa_1, \dots, \kappa_n. c$ , where all the refinement variables in  $c$  are existentially bound at the top. A *verification condition* (VC) is a closed constraint that has *no* Horn variables.

The running example  $c_0$  is itself a closed constraint. In the sequel we abuse notation and write  $c$  for both open and closed constraints, as the intended meaning is always clear from context.

**Predicates and Interpretations** A *predicate* in  $\mathcal{P}$  is a LEAN term of sort  $b_1 \rightarrow \dots \rightarrow b_n \rightarrow \text{Prop}$  (i.e.,  $\mathcal{P} \subset \mathcal{T}$ ). For a horn variable  $\kappa$ , an *interpretation*  $\sigma \in \mathcal{P}$  is a predicate of the same type as  $\kappa$ . For  $c_0$ , the predicate  $\sigma_1 \doteq \lambda v x. v \geq 1$  is an interpretation for  $\kappa_1$ , since  $\sigma_1 : \text{Int} \rightarrow \text{Int} \rightarrow \text{Prop}$ .

**Satisfiability** We *apply* an interpretation  $\sigma$  to a constraint  $c$  by replacing each  $\kappa$ -application  $\kappa(\bar{t})$  with  $\sigma(\bar{t})$ ; we write  $c[\kappa := \sigma]$  for the result. Applying an interpretation for each  $\kappa$  in a closed constraint instantiates the existentials, leaving a  $\kappa$ -free proposition. A constraint  $c$  is *satisfiable* if there exist an interpretation for each  $\kappa$  such that applying them makes the constraint a valid proposition (i.e., provable in LEAN). If an interpretation makes a constraint satisfiable we call it a *solution*. For example, applying  $\sigma_1$  to  $c_0$  (and  $\beta$ -reducing) replaces  $\kappa_1(v, x)$  with  $v \geq 1$  and  $\kappa_1(a, x)$  with  $a \geq 1$  leaving the following verification condition:

$$\forall x. 0 \leq x \rightarrow (\forall v. v = x + 1 \rightarrow v \geq 1) \wedge (\forall a. a \geq 1 \rightarrow 1 \leq a)$$

This VC is valid; hence  $\sigma_1$  is a solution of  $\kappa_1$  and  $c_0$  is satisfiable.

**Theories for free** As terms and predicates are LEAN terms, they can use any theory expressible in LEAN. For example, arithmetic inequalities  $x \leq y$  over  $\text{Int}$  sorted variables  $x$  and  $y$ , mask equations over  $\text{BitVec } n$ , or equations over user-defined functions  $v = \text{fib}(i)$ .

### 3.2 Structural Operations

To solve a constraint, the solvers in §4 need to know, for each Horn variable, the smallest enclosing scope and the hypotheses visible there; we now define the machinery to compute this.

**Prefixes, Contexts, Heads** A *prefix*  $\rho$  is a list of *steps* used to navigate a path through a constraint's syntax tree. A *step*  $s$  is either a left L or right R choice in a conjunction, a variable binding B, or an assumption G. A *context*  $\Gamma$  is an ordered list of variable bindings  $x : b$  (one per  $\forall$ ), named hypotheses  $h : p$  (one per  $\rightarrow$ ), or a L or R choice gathered along a prefix.

$$\begin{aligned} \textbf{Prefixes} \quad \rho &::= \varepsilon \mid s \cdot \rho \\ \textbf{Steps} \quad s &::= \text{L} \mid \text{R} \mid \text{B} \mid \text{G} \\ \textbf{Contexts} \quad \Gamma &::= \varepsilon \mid x : b ; \Gamma \mid h : p ; \Gamma \mid \text{L} ; \Gamma \mid \text{R} ; \Gamma \end{aligned}$$

The procedure  $c \uparrow \rho$  (Fig. 9) computes the *context* of a constraint  $c$  at prefix  $\rho$ , by accumulating the bindings and guards along  $\rho$ . The procedure  $c \downarrow \rho$  (Fig. 9) returns the sub-constraint of  $c$  at the end of  $\rho$ . The *truncated context*  $\Gamma \setminus \rho$  is the context obtained by dropping the prefix of  $\Gamma$  that matches  $\rho$ .

**Goals** A prefix  $\rho$  is a *head-prefix* of  $c$  if  $c \downarrow \rho$  is a head atom  $p$ . We write  $\text{heads}(c)$  for the set of Horn variables with a *head* application in  $c$ , i.e.,  $\text{heads}(c) \doteq \{\kappa \mid \kappa(\bar{t}) = c \downarrow \rho \text{ for some head-prefix } \rho\}$ . A *goal*  $\Gamma \vdash p$  is a context  $\Gamma$  paired with a head  $p$ . For a head-prefix  $\rho$ , the goal of  $c$  at  $\rho$  is defined as  $\text{Goal}(c, \rho) \doteq c \uparrow \rho \vdash c \downarrow \rho$ . For example, in the running constraint  $c_0$ , we have:

$$\text{Goal}(c_0, \text{B} \cdot \text{G} \cdot \text{L} \cdot \text{B} \cdot \text{G}) = x : \text{Int}; h_0 : 0 \leq x; \text{L}; v : \text{Int}; h_1 : v = x + 1 \vdash \kappa_1(v, x)$$

Given a context  $\Gamma$  and term  $\varphi$ , we say the goal  $\Gamma \vdash \varphi$  is *valid* if  $\varphi$  is derivable under the bindings and guards in  $\Gamma$ . A goal is a  $\kappa$ -*goal* if its head is  $\kappa(\bar{t})$ ; an interpretation  $\sigma$  is a solution for such a goal if the goal  $\Gamma \vdash \sigma(\bar{t})$  is valid. For example, the interpretation  $\sigma_1 \doteq \lambda v x. v \geq 1$  is a solution for the goal above, since  $v \geq 1$  is valid under the assumptions  $0 \leq x$  and  $v = x + 1$ .

**Scopes and Well-formedness** The procedure  $\text{Scope}(\kappa, c)$ , shown in Fig. 9 computes the *scope* of a variable  $\kappa$  in a constraint  $c$ . Informally,  $\text{Scope}(\kappa, c)$  returns a prefix  $\rho$  corresponding to a path from the root of  $c$  to the *smallest sub-constraint* of  $c$  that contains *all* the applications of  $\kappa$  in  $c$ . The scope is more interesting once a constraint has more than one Horn variable, so consider extending  $c_0$

| Scope : $(\mathcal{K} \times C) \rightarrow \text{Prefix}$            | $(\downarrow) : (C \times \text{Prefix}) \rightarrow C$               | $(\uparrow) : (C \times \text{Prefix}) \rightarrow \text{Ctx}$        |
|-----------------------------------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------|
| Scope( $\kappa, \forall x:b. c$ ) $\doteq$ B · Scope( $\kappa, c$ )   | $(\forall x:b. c) \downarrow B \cdot \rho \doteq c \downarrow \rho$   | $(\forall x:b. c) \uparrow B \cdot \rho \doteq x:b; c \uparrow \rho$  |
| Scope( $\kappa, c_1 \wedge c_2$ )                                     | $(p \rightarrow c) \downarrow G \cdot \rho \doteq c \downarrow \rho$  | $(p \rightarrow c) \uparrow G \cdot \rho \doteq h:p; c \uparrow \rho$ |
| $\kappa \in c_1, \kappa \notin c_2 \doteq$ L · Scope( $\kappa, c_1$ ) | $(c_1 \wedge c_2) \downarrow L \cdot \rho \doteq c_1 \downarrow \rho$ | $(c_1 \wedge c_2) \uparrow L \cdot \rho \doteq L; c_1 \uparrow \rho$  |
| $\kappa \notin c_1, \kappa \in c_2 \doteq$ R · Scope( $\kappa, c_2$ ) | $(c_1 \wedge c_2) \downarrow R \cdot \rho \doteq c_2 \downarrow \rho$ | $(c_1 \wedge c_2) \uparrow R \cdot \rho \doteq R; c_2 \uparrow \rho$  |
| Scope( $\kappa, p \rightarrow c$ )                                    | $c \downarrow \varepsilon \doteq c$                                   | $c \uparrow \varepsilon \doteq \varepsilon$                           |
| $\kappa \notin p \doteq$ G · Scope( $\kappa, c$ )                     |                                                                       |                                                                       |
| Scope( $\kappa, c$ ) $\doteq$ $\varepsilon$                           |                                                                       |                                                                       |

Fig. 9. Scope( $\kappa, c$ ) is the scope of a  $\kappa$  in  $c$ ;  $c \downarrow \rho$  is the *head* of  $c$  at  $\rho$  and  $c \uparrow \rho$  is the *context* of  $c$  at  $\rho$ . In  $\uparrow$ , each guard hypothesis gets a fresh name  $h$ .

with a second variable  $\kappa_2$  and a sub-constraint nested under the guard  $\kappa_1(a, x)$ . For the constraint  $c_1$  below, we get the scopes shown on the right ( $\wedge$  is right associative).

$$\begin{array}{ll}
 \exists \kappa_2, \kappa_1. \forall x. 0 \leq x \rightarrow & \\
 \quad \forall v. v = x + 1 \rightarrow \kappa_1(v, x) & \\
 c_1 \doteq \quad \wedge \forall a. \kappa_1(a, x) \rightarrow & \text{Scope}(\kappa_1, c_1) \doteq B \cdot G \\
 \quad 1 \leq a & \text{Scope}(\kappa_2, c_1) \doteq B \cdot G \cdot R \cdot B \cdot G \cdot R \\
 \quad \wedge \kappa_2(a + 1, a, x) & \\
 \quad \wedge \forall v. \kappa_2(v, a, x) \rightarrow 2 \leq v & \\
 & (2)
 \end{array}$$

**Well-formedness** The solver in §4 assumes that each Horn variable’s arguments are passed consistently with the binders in scope; we call this property well-formedness. Formally, let  $\rho \doteq$  Scope( $\kappa, c$ ),  $\Gamma \doteq c \uparrow \rho$ , and  $\text{dom}(\Gamma) \doteq x_0, \dots, x_n$  the binders of  $\Gamma$  listed innermost first. We say that  $c$  is *well-formed* for  $\kappa$  if every application of  $\kappa$  in  $c$  has the form  $\kappa(\dots, x_0, \dots, x_n)$ , i.e., its trailing arguments are exactly the binders  $\text{dom}(\Gamma)$  in scope. Its remaining leading arguments are arbitrary terms, we write  $\text{rank}(\kappa)$  for their number. A constraint  $c$  is *well-formed* if it is well-formed for all  $\kappa \in \text{KVars}(c)$ . It is straightforward to ensure well-formedness by simply *padding* the parameters of each  $\kappa$ . Thus, in the sequel, for simplicity, we assume that all constraints are well-formed. Constraint  $c_1$  in Eq. (2) is well-formed as  $x$  is the only binder in the scope of  $\kappa_1$ , and each application of  $\kappa_1$  has  $x$  as argument, while the binders in the scope of  $\kappa_2$  are  $a$  and  $x$ , and each application of  $\kappa_2$  has them as arguments.

### 3.3 Proof Terms

The solvers in §4 emit proofs to justify their rewrites. Fig. 8 shows the fragment of LEAN’s terms that the solvers emit as proofs. The fragment comprises exactly the introduction and elimination forms of the constraint logic (Fig. 8) and the disjunctions and existentials that arise in the solutions  $\sigma$  computed by our solvers. Specifically, the fragment contains terms  $t$ ;  $\lambda$ -terms and applications for introducing and eliminating  $\forall$  and implication; pairs  $\langle q, q' \rangle$  and projections  $q.1, q.2$  for conjunction, together with  $\text{inl}$  and  $\text{inr}$  for disjunction; existential witnesses  $\langle t, q \rangle$ , unpacking terms  $\text{let } \langle x, h \rangle = q \text{ in } q'$ , and the constants  $\text{rfl}$  and  $\langle \rangle$  for equality and  $\top$ . The typing of these terms is inherited from the LEAN kernel, which re-checks every certificate our solvers emit. These are the standard notations for LEAN’s logical connectives, and the same core is shared by other type-theoretic proof assistants such as Rocq [69] and AGDA [57], so the certifying solvers of §4 are not tied to LEAN.

## 4 Solvers

FLEX uses a three-step approach to solving Horn constraints. First, we *partition* the horn variables into a cut set and an acyclic set (§ 4.1). Second, we *eliminate* the acyclic variables by substituting them with their strongest solutions (§ 4.2). Third, we perform a *fixpoint* computation to find a conservative approximation for the remaining cyclic variables (§ 4.4). These steps are implemented in two LEAN tactics — Zap and Fix — which *rewrite* the original horn constraint by eliminating the acyclic and cut variables respectively, leaving behind a  $\kappa$ -free VC which can be discharged using LEAN tactics like simp, omega or SMT-inspired ones like grind. While the algorithms used in Zap and Fix are inspired by prior work, in the foundational setting they must also *certify* the soundness of the rewrites. Next, we describe how FLEX implements partitioning (§ 4.1), and the two tactics, Zap (§ 4.2) and Fix (§ 4.4), in a certifying fashion, to obtain a foundational CHC solver.

### 4.1 Partitioning Horn Variables

**Dependencies** The *dependencies* of a constraint  $c$  is a subset of  $\mathcal{K} \times \mathcal{K}$  defined by the function

$$\text{Dep}(c) \doteq \{(\kappa, \kappa') \mid \exists \rho. \kappa \in c \uparrow \rho \text{ and } \kappa'(\bar{t}) = c \downarrow \rho\}$$

We say  $\kappa'$  *depends on*  $\kappa$  in  $c$  if  $(\kappa, \kappa') \in \text{Dep}(c)$ . Informally, this means that  $\kappa$  occurs in a guard  $p$  along some path to a head containing  $\kappa'$ . The *dependency graph* of a constraint  $c$  is the directed graph  $\text{Graph}(c)$  whose vertices are  $\text{KVars}(c)$  and which has an edge from  $\kappa$  to  $\kappa'$  if  $\kappa'$  depends on  $\kappa$  in  $c$ . Intuitively, if  $\kappa'$  depends on  $\kappa$ , then we need to solve for  $\kappa$  *before* we can solve for  $\kappa'$ .

**Cycles, Cuts and Partitions** A set of variables  $\bar{\kappa} \subseteq \text{KVars}(c)$  is a *cut set* for  $c$  if it corresponds to a *feedback vertex set* of  $\text{Graph}(c)$ , i.e., if removing the  $\bar{\kappa}$  vertices and their incident edges renders  $\text{Graph}(c)$  acyclic [40]. We say that  $\bar{\kappa}_C, \bar{\kappa}_A$  *partitions* the horn variables of a constraint  $c$ , if (1)  $\bar{\kappa}_C$  and  $\bar{\kappa}_A$  are disjoint, (2)  $\bar{\kappa}_C \cup \bar{\kappa}_A = \text{KVars}(c)$ , (3)  $\bar{\kappa}_C$  is a cut set for  $c$ . We say informally that  $\bar{\kappa}_A$  is the set of *acyclic* variables. We define  $\text{Partition}(\bar{\kappa}, c)$  to be a function that computes the dependencies of  $c$ , builds a dependency graph, and computes a feedback vertex set to return a partition of the horn variables of  $c$ . The acyclic set  $\bar{\kappa}_A$  is returned in *topological* order: each variable is bound outside the variables it depends on, so the innermost-first elimination of § 4.2 removes a variable's dependencies before the variable itself, and every computed solution mentions only cut variables. Computing a *minimum* feedback vertex set is NP-complete [40], so  $\text{Partition}(\bar{\kappa}, c)$  uses a greedy heuristic based on Tarjan's algorithm [68]; minimality affects only the residual CHC size, not soundness.

### 4.2 Solving Acyclic Variables

**Top-level Algorithm (Zap):** Fig. 10 defines the top-level Zap algorithm which takes as input a closed constraint  $\exists \bar{\kappa}. c$ , and returns a pair of a *new* constraint  $\exists \bar{\kappa}_C. c'$  where all occurrences of the acyclic variables have been removed, and a *proof term*  $q'$  that shows that the output constraint *entails* the original constraint. First, Zap partitions the horn variables  $\bar{\kappa}$  into a cut set  $\bar{\kappa}_C$  and an acyclic set  $\bar{\kappa}_A$ . Next, Zap invokes Zaps, which recurses over the acyclic prefix: it peels the outermost  $\kappa$ , recursively eliminates the inner variables to obtain a residual  $c_0$ , and invokes Zap1 to eliminate  $\kappa$  from  $c$ . Zaps then composes the proofs  $q_2 : c_2 \rightarrow \exists \kappa. c_1$  and  $q_1 : c_1 \rightarrow c$  into a proof  $q_3 : c_2 \rightarrow \exists \kappa. c$  by unpacking and repacking the  $\kappa$ -witness. At the top, Zap re-bundles the cut and acyclic witnesses in the original binder order of  $\bar{\kappa}$ . The Zap1 function eliminates a *single* acyclic variable  $\kappa$  from a constraint  $c$  in two steps. In the first step, Zap1 invokes the Elim procedure to *eliminate* the variable  $\kappa$  by computing a triple comprising (1) the *strongest solution*  $\sigma$  for  $\kappa$  in  $c$ , (2) a prefix  $\rho$  corresponding to the scope of  $\kappa$  in  $c$ , and (3) a constraint  $c'$  where  $\kappa$  has been eliminated. In the second, Zap1 invokes  $\text{Cert}_{\sigma, \rho}^\kappa$  with the triple to construct the term  $\lambda h. \langle \sigma, \text{Cert}_{\sigma, \rho}^\kappa(\varepsilon, h, c) \rangle$  that

|     |                                                                                                                                                |     |                                                                                                                    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------|-----|--------------------------------------------------------------------------------------------------------------------|
| 589 | $\text{Zap} : C \rightarrow (C \times \text{Proof})$                                                                                           | 589 | $\text{Elim} : (\mathcal{K} \times C) \rightarrow (\mathcal{P} \times \text{Prefix} \times C)$                     |
| 590 | $\text{Zap}(\exists \bar{\kappa}. c) \doteq (\exists \bar{\kappa}_C. c', q' : \exists \bar{\kappa}_C. c' \rightarrow \exists \bar{\kappa}. c)$ | 590 | $\text{Elim}(\kappa, c) \doteq (\sigma, \rho, c')$                                                                 |
| 591 | <b>where</b>                                                                                                                                   | 591 | <b>where</b>                                                                                                       |
| 592 | $(\bar{\kappa}_C, \bar{\kappa}_A) \doteq \text{Partition}(\bar{\kappa}, c)$                                                                    | 592 | $\rho \doteq \text{Scope}(\kappa, c)$                                                                              |
| 593 | $(c', q) \doteq \text{Zaps}(\exists \bar{\kappa}_A. c)$                                                                                        | 593 | $\sigma \doteq \lambda \bar{z}. \lambda \bar{x}. \text{Sol}(\kappa, c \downarrow \rho)$                            |
| 594 | $q' \doteq \lambda h_1. \text{let } \langle \bar{\kappa}_C, h_2 \rangle = h_1 \text{ in}$                                                      | 594 | $c' \doteq \text{Red}(\kappa, \sigma, c)$                                                                          |
| 595 | $\text{let } \langle \bar{\kappa}_A, h_3 \rangle = q h_2 \text{ in}$                                                                           | 595 | $\bar{z} \doteq z_0, \dots, z_{n-1} \text{ where } n = \text{rank}(\kappa)$                                        |
| 596 | $\langle \bar{\kappa}, h_3 \rangle$                                                                                                            | 596 | $\bar{x} \doteq \text{dom}(c \uparrow \rho)$                                                                       |
| 597 |                                                                                                                                                | 597 |                                                                                                                    |
| 598 | $\text{Zaps} : C \rightarrow (C \times \text{Proof})$                                                                                          | 598 | $\text{Red} : (\mathcal{K} \times \mathcal{P} \times C) \rightarrow C$                                             |
| 599 | $\text{Zaps}(\exists \kappa. c) \doteq (c_2, q_3 : c_2 \rightarrow \exists \kappa. c)$                                                         | 599 | $\text{Red}(\kappa, \sigma, c_1 \wedge c_2) \doteq \text{Red}(\kappa, \sigma, c_1) \wedge$                         |
| 600 | <b>where</b>                                                                                                                                   | 600 | $\text{Red}(\kappa, \sigma, c_2)$                                                                                  |
| 601 | $(c_1, q_1) \doteq \text{Zaps}(c)$                                                                                                             | 601 | $\text{Red}(\kappa, \sigma, \forall x : b. c) \doteq \forall x : b. \text{Red}(\kappa, \sigma, c)$                 |
| 602 | $(c_2, q_2) \doteq \text{Zap1}(\kappa, c_1)$                                                                                                   | 602 | $\text{Red}(\kappa, \sigma, p \rightarrow c) \doteq p[\kappa := \sigma] \rightarrow \text{Red}(\kappa, \sigma, c)$ |
| 603 | $q_3 \doteq \lambda h_1. \text{let } \langle \kappa, h_2 \rangle = q_2 h_1 \text{ in}$                                                         | 603 | $\text{Red}(\kappa, \sigma, \kappa(\bar{t})) \doteq \top$                                                          |
| 604 | $\langle \kappa, q_1 h_2 \rangle$                                                                                                              | 604 | $\text{Red}(\kappa, \sigma, p) \doteq p$                                                                           |
| 605 | $\text{Zaps}(c) \doteq (c, \lambda h. h)$                                                                                                      | 605 |                                                                                                                    |
| 606 | $\text{Zap1} : (\mathcal{K} \times C) \rightarrow (C \times \text{Proof})$                                                                     | 606 | $\text{Sol} : (\mathcal{K} \times C) \rightarrow \mathcal{T}$                                                      |
| 607 | $\text{Zap1}(\kappa, c) \doteq (c', q : c' \rightarrow \exists \kappa. c)$                                                                     | 607 | $\text{Sol}(\kappa, c)$                                                                                            |
| 608 | <b>where</b>                                                                                                                                   | 608 | $\mid \kappa \notin \text{heads}(c) \doteq \perp$                                                                  |
| 609 | $(\sigma, \rho, c') \doteq \text{Elim}(\kappa, c)$                                                                                             | 609 | $\text{Sol}(\kappa, c_1 \wedge c_2) \doteq \text{Sol}(\kappa, c_1) \vee \text{Sol}(\kappa, c_2)$                   |
| 610 | $q \doteq \lambda h. \langle \sigma, \text{Cert}_{\sigma, \rho}^{\kappa}(\varepsilon, h, c) \rangle$                                           | 610 | $\text{Sol}(\kappa, \forall x : b. c) \doteq \exists x : b. \text{Sol}(\kappa, c)$                                 |
| 611 |                                                                                                                                                | 611 | $\text{Sol}(\kappa, p \rightarrow c) \doteq p \wedge \text{Sol}(\kappa, c)$                                        |
| 612 |                                                                                                                                                | 612 | $\text{Sol}(\kappa, \kappa(\bar{t})) \doteq \bigwedge_{i < \text{rank}(\kappa)} z_i = t_i$                         |

Fig. 10. Zap eliminates acyclic horn variables (left) using Elim to compute solutions (right). The guarded first equation of Sol collapses sub-constraints without  $\kappa$  head applications to  $\perp$ , so the solution is  $\kappa$ -free, and all its free variables are among the  $\bar{z}, \bar{x}$  bound in Elim.

|     |                                                   |     |                                                   |     |                                           |
|-----|---------------------------------------------------|-----|---------------------------------------------------|-----|-------------------------------------------|
| 617 | $\exists \kappa_2. \kappa_1.$                     | 617 | $\exists \kappa_2.$                               | 617 | $\forall x. 0 \leq x \rightarrow$         |
| 618 | $\forall v. 0 \leq x \rightarrow$                 | 618 | $\forall v. v = x + 1 \rightarrow \top$           | 618 | $\forall v. v = x + 1 \rightarrow \top$   |
| 619 | $\forall v. v = x + 1 \rightarrow \kappa_1(v, x)$ | 619 | $\forall v. v = x + 1 \rightarrow \top$           | 619 | $\forall v. v = x + 1 \rightarrow \top$   |
| 620 | $\wedge \forall a. \kappa_1(a, x) \rightarrow$    | 620 | $\wedge \forall a. a = x + 1 \rightarrow$         | 620 | $\wedge \forall a. a = x + 1 \rightarrow$ |
| 621 | $1 \leq a$                                        | 621 | $1 \leq a$                                        | 621 | $1 \leq a$                                |
| 622 | $\wedge \kappa_2(a + 1, a, x)$                    | 622 | $\wedge \kappa_2(a + 1, a, x)$                    | 622 | $\wedge \top$                             |
| 623 | $\wedge \forall v. \kappa_2(v, a, x) \rightarrow$ | 623 | $\wedge \forall v. \kappa_2(v, a, x) \rightarrow$ | 623 | $\wedge \forall v. v = a + 1 \rightarrow$ |
| 624 | $2 \leq v$                                        | 624 | $2 \leq v$                                        | 624 | $2 \leq v$                                |

Fig. 11. Eliminating acyclic variables with Zap. (L) initial constraint from Eq. (2), (C) result of reducing  $\kappa_1$ , and (R) result of reducing  $\kappa_2$  yielding a VC with no Horn variables.

certifies the elimination (§ 4.3): it maps any proof  $h$  of  $c'$  to a proof of  $\exists \kappa. c$ , with  $\sigma$  as the witness. Fig. 11 illustrates this process on the constraint  $c_1$  from Eq. (2).

**Variable Elimination (Elim):** The procedure  $\text{Elim}(\kappa, c)$ , shown in Fig. 10, eliminates  $\kappa$  from  $c$  in three steps. First, it computes the *scope*  $\rho$  of  $\kappa$  in  $c$ . Next, it calls Sol on  $c \downarrow \rho$  — the sub-constraint of  $c$  where  $\kappa$  occurs — to compute  $\sigma$ : the *strongest solution* for  $\kappa$ . Finally, the procedure invokes  $\text{Red}(\kappa, \sigma, c)$ , to compute the *reduced constraint* where all *head* applications of  $\kappa$  are replaced by  $\top$  — as they are guaranteed to hold by virtue of how  $\sigma$  was computed — and all other non-head occurrences of  $\kappa(\bar{t})$  are replaced by the solution  $\sigma(\bar{t})$ , yielding a constraint *without*  $\kappa$ .

| $\text{Cert}_{\sigma,\rho}^{\kappa} : (\text{Ctx} \times \text{Proof} \times C) \rightarrow \text{Proof}$                                                                                                             | $\text{Nav} : (\text{Ctx} \times \mathcal{P}) \rightarrow \text{Proof}$                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| $\text{Cert}_{\sigma,\rho}^{\kappa}(\Gamma, q, \forall x:b. c) \doteq \lambda x. \text{Cert}_{\sigma,\rho}^{\kappa}(\Gamma; x:b, q \ x, c)$                                                                           | $\text{Nav}(\text{L}; \Gamma, p_L \vee p_R) \doteq \text{inl } \text{Nav}(\Gamma, p_L)$                 |
| $\text{Cert}_{\sigma,\rho}^{\kappa}(\Gamma, q, p \rightarrow c) \doteq \lambda h_p. \text{Cert}_{\sigma,\rho}^{\kappa}(\Gamma; h_p:p, q \ h_p, c)$                                                                    | $\text{Nav}(\text{R}; \Gamma, p_L \vee p_R) \doteq \text{inr } \text{Nav}(\Gamma, p_R)$                 |
| $\text{Cert}_{\sigma,\rho}^{\kappa}(\Gamma, q, c_1 \wedge c_2) \doteq \langle \text{Cert}_{\sigma,\rho}^{\kappa}(\Gamma; \text{L}, q.1, c_1), \text{Cert}_{\sigma,\rho}^{\kappa}(\Gamma; \text{R}, q.2, c_2) \rangle$ | $\text{Nav}(x:b; \Gamma, \exists z:b. p) \doteq \langle x, \text{Nav}(\Gamma, p[z := x]) \rangle$       |
| $\text{Cert}_{\sigma,\rho}^{\kappa}(\Gamma, q, \kappa(\bar{t})) \doteq \text{Nav}(\Gamma \setminus \rho, \sigma(\bar{t}))$                                                                                            | $\text{Nav}(q; \Gamma, p \wedge p') \doteq \langle q, \text{Nav}(\Gamma, p') \rangle$                   |
| $\text{Cert}_{\sigma,\rho}^{\kappa}(\Gamma, q, p) \doteq q$                                                                                                                                                           | $\text{Nav}(\varepsilon, (\bigwedge_i z_i = t_i)) \doteq \langle \text{rfl}, \dots, \text{rfl} \rangle$ |
|                                                                                                                                                                                                                       | $\text{Nav}(\varepsilon, \top) \doteq \langle \rangle$                                                  |

Fig. 12.  $\text{Cert}_{\sigma,\rho}^{\kappa}$  transforms a proof of the reduced constraint into a proof of the original, invoking  $\text{Nav}$  at each divergent  $\kappa$ -head to rebuild a proof of  $\sigma(\bar{t})$  by replaying the truncated context against the  $\vee/\exists/\wedge$  structure of the solution  $\sigma$ .

**Strongest Solution (Sol):** A solution  $\sigma$  is *valid* for  $\kappa$  in  $c$  if it is valid for every  $\kappa$ -goal (defined in §3). The procedure  $\text{Sol}(\kappa, c)$ , shown in Fig. 10, computes the *body* of such a solution: the *disjunction* of the contexts at each head-prefix for  $\kappa$ , existentially quantifying the variables not in the scope;  $\text{Elim}$  closes the body under  $\bar{z}$  and  $\bar{x}$  to obtain  $\sigma$ . The guard  $\kappa \notin \text{heads}(c)$  collapses each sub-constraint without a  $\kappa$  head to an inert  $\perp$ , keeping dead-branch horn applications out of the solution. For example, Fig. 11 shows below each arrow, the solution computed for the  $\kappa$  reduced in the respective step. In the sub-constraint for  $\kappa_1$  there is exactly one head application, with the (truncated) context  $v: \text{Int}; v = x + 1$ ; the conjunct with the *uses* of  $\kappa_1$  collapses to  $\perp$ . Thus, the computed solution is  $\lambda z_0 x. (\exists v. v = x + 1 \wedge z_0 = v) \vee \perp$  which we simplify to  $\lambda z_0 x. z_0 = x + 1$ . As such, we can check that this procedure returns the *strongest solution* for  $\kappa$  in  $c$ , meaning, it returns a solution  $\sigma$  that entails every other valid solution for  $\kappa$  in  $c$ .

**Scope avoids Exponential Blowup** The procedure  $\text{Elim}$  is analogous to the similarly named one from [14], which introduces the scope optimization to sidestep an exponential blowup that otherwise occurs in practice. Specifically, if we compute  $\text{Sol}(\kappa, c)$  *without* considering the scope  $\rho$ , then the assumptions from  $c \upharpoonright \rho$  get (unnecessarily) included in the strongest solution, which causes an exponential duplication of assumptions as we eliminate multiple variables.

### 4.3 Certification

In a foundational setting, one does not simply rewrite a constraint. The `LEAN` kernel, ultimately, demands a proof of the original constraint, not the rewritten one. To this end, `Zap` uses  $\text{Cert}_{\sigma,\rho}^{\kappa}$  to produce a term that *converts* any proof of the rewritten constraint into a proof of the original one.

**Certifying an Assignment ( $\text{Cert}_{\sigma,\rho}^{\kappa}$ ):** The procedure  $\text{Cert}_{\sigma,\rho}^{\kappa}(\Gamma, q, c)$  (Fig. 12) takes two sets of inputs. First, a parametric set comprising  $\kappa$ , its solution  $\sigma$  and its scope  $\rho$  in  $c$ . Second, a set with a context  $\Gamma$  of accumulated bindings, the original sub-constraint  $c$ , and a proof  $q$  of the  $\sigma$ -reduced sub-constraint (i.e.,  $c' \doteq \text{Red}(\kappa, \sigma, c)$ ). The procedure then traverses  $c$ , transforming  $q$  at each step into a proof of the corresponding sub-constraint. For the  $\forall x:b. c$  (resp.  $p \rightarrow c$ ) case, the procedure emits a  $\lambda$ -term that applies the supplied input  $x$  (resp.  $h_p$ ) to  $q$  to recursively generate the certificate for the sub-constraint  $c$ . For a  $c_1 \wedge c_2$  the procedure emits a pair term comprising the sub-terms obtained from the proofs of  $c_1$  and  $c_2$  constructed using the proofs  $q.1$  and  $q.2$  respectively. The original and reduced constraints diverge only when the original head is of the form  $\kappa(\bar{t})$  but the reduced head is  $\top$ ; in that case,  $q$  is vacuous. Other heads  $p$  are unchanged so we can directly reuse the corresponding proof term  $q$ .

**Reconciling divergent heads ( $\text{Nav}$ ):** At the divergent heads in  $\text{Cert}_{\sigma,\rho}^{\kappa}$  we call  $\text{Nav}$  to furnish us with a proof for the original goal  $\kappa(\bar{t})$  post-substitution, i.e., for  $\sigma(\bar{t})$ . Recall that the solution  $\sigma$  was

a disjunction of contexts at each head-prefix for  $\kappa$  in the sub-constraint  $c \downarrow \rho$  where  $\kappa$  occurs. Thus, we use the *truncated* context  $(\Gamma \setminus \rho)$  that led to the divergent head to *reconstruct* the proof term for that disjunction from the context. Nav replays the truncated context against the structure of  $\sigma$ , whose  $\forall/\exists$  shape mirrors the scoped sub-constraint. Each context entry selects the matching introduction form: a branch selects *inl* or *inr*, a bound variable supplies an existential witness, and a guard hypothesis supplies a conjunct. The descent bottoms out at the slot equalities  $\bigwedge_i z_i = t_i$ , using *rfl* to discharge them.

**Example** In the example in Fig. 11,  $\kappa_1$  has the scope prefix  $\rho \doteq B \cdot G$ . Thus, at the head  $\kappa_1(v, x)$ ,  $\text{Cert}_{\sigma, \rho}^{\kappa}$  has accumulated the context  $\Gamma \doteq x : \text{Int}; h_0 : 0 \leq x; L; v : \text{Int}; h_1 : v = x + 1$  (naming the guard hypotheses  $h_0, h_1$ ), and the truncated context is  $\Gamma \setminus \rho \doteq L; v : \text{Int}; h_1 : v = x + 1$ . Given the solution  $\sigma \doteq \lambda z_0 x. (\exists v'. v' = x + 1 \wedge z_0 = v') \vee \perp$  (the bound variable renamed to  $v'$ ), Nav replays the path, as follows, consuming one entry per node:

$$\begin{aligned}
 & \text{Nav}(L; v : \text{Int}; h_1 : v = x + 1, \sigma(v, x)) \\
 = & \text{inl } \text{Nav}(v : \text{Int}; h_1 : v = x + 1, \exists v'. v' = x + 1 \wedge v = v') && L \text{ picks the live disjunct} \\
 = & \text{inl } \langle v, \text{Nav}(h_1 : v = x + 1, (v = x + 1) \wedge v = v) \rangle && v \text{ witnesses } \exists v' \\
 = & \text{inl } \langle v, \langle h_1, \text{Nav}(\varepsilon, v = v) \rangle \rangle && h_1 \text{ discharges } v = x + 1 \\
 = & \text{inl } \langle v, \langle h_1, \text{rfl} \rangle \rangle. && \text{slot equality is rfl}
 \end{aligned}$$

**Correctness** The correctness of Zap is stated via two theorems. The first one, from [14], says that  $\text{Zap}(c)$  returns a constraint  $c'$ , without any acyclic horn variables, that is logically equivalent to  $c$ .

**Theorem 4.1** (Zap-Equivalence). [14] *If  $(c', \cdot) = \text{Zap}(c)$  then  $c$  is satisfiable iff  $c'$  is satisfiable.*

In the foundational setting, we also crucially depend on the following soundness result, which says that the proof returned by  $\text{Zap}(c)$  explicitly witnesses the correctness of the rewrite, *i.e.*, it allows us to convert any proof of  $c'$  into one of  $c$ :

**Theorem 4.2** (Zap-Soundness). *If  $(c', q) = \text{Zap}(c)$  then  $\vdash q : c' \rightarrow c$ .*

#### 4.4 Solving Cyclic Variables

Next, let us see how the *Fix* tactic solves the cyclic variables by *predicate abstraction* [28], which works by solving each  $\kappa$  to a conjunction of predicates drawn from a library of user-supplied *qualifiers*. Hitherto, predicate abstraction has typically been carried out using SMT solvers [21, 62], where it is limited to whatever theories the SMT engine can handle efficiently. The foundational setting provides two significant benefits. First, we can use arbitrary LEAN *predicates* as qualifiers, and, correspondingly, second, we can use arbitrary LEAN automation (including, of course, SMT-based tactics) to implement user-definable *oracles* that can allow synthesizing invariants over arbitrary theories. Next, we illustrate the above by describing how *Fix* ticks.

**Qualifiers, Instances and Candidates** The predicate abstraction procedure *PredAbs* uses a set of qualifiers to compute a solution for each  $\kappa$ . Informally, qualifiers are predicate *templates*: *PredAbs* computes solutions by *conjoining* all *valid* instances of the templates. Formally, a *qualifier*  $q$  is a LEAN predicate of the form  $q \doteq \lambda z_0 : b_0, \dots, z_n : b_n. t$ . Let  $\kappa : b'_0 \rightarrow \dots \rightarrow b'_m \rightarrow \text{Prop}$  be a Horn variable. We say that  $\iota$  is a  $\kappa/q$  *instance* if (1)  $\iota$  is a map  $[0, n] \rightarrow [0, m]$ , where (2) for each  $0 \leq j \leq n$ , we have  $b_j = b'_{\iota(j)}$ . That is, an instance  $\iota$  maps each parameter of the qualifier to a parameter of the  $\kappa$  of the same sort. A *candidate* is a pair of a qualifier  $q$  and an instance  $\iota$  written as  $q_\iota$ . We overload  $q_\iota$  to also denote the predicate it induces by instantiating  $q$  along  $\iota$ , namely  $q_\iota \doteq \lambda z_0 \dots z_m. q(z_{\iota(0)}, \dots, z_{\iota(n)})$ . Given a *candidate set*  $\overline{q}_\iota$ , we write  $\bigwedge \overline{q}_\iota$  for its pointwise conjunction.

|     |                                                                                                                                                         |                                                                                                                                                                           |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 736 | $\text{Fix} : (C \times Q) \rightarrow (C \times \text{Proof})$                                                                                         | $\text{PredAbs} : (C \times Q) \rightarrow \mathcal{A}$                                                                                                                   |
| 737 | $\text{Fix}(\exists \overline{\kappa}_C. c, Q) \doteq (c', q : c' \rightarrow \exists \overline{\kappa}_C. c)$                                          | $\text{PredAbs}(\exists \overline{\kappa}_C. c, Q) \doteq \text{Fixpoint}(c, \theta)$                                                                                     |
| 738 | <b>where</b>                                                                                                                                            | <b>where</b>                                                                                                                                                              |
| 739 | $\theta \doteq \text{PredAbs}(\exists \overline{\kappa}_C. c, Q)$                                                                                       | $\theta \doteq [\kappa \mapsto \text{Init}(\kappa, Q) \mid \kappa \in \overline{\kappa}_C]$                                                                               |
| 740 | $c' \doteq \text{Red}^*(\overline{\kappa}_C, c)$                                                                                                        | $\text{Init}(\kappa, Q) \doteq \{q_i \mid q \in Q, i \in \kappa/q \text{ instances}\}$                                                                                    |
| 741 | $q \doteq \lambda h. \langle \bigwedge \theta(\kappa_C), \text{Cert}_\theta(\varepsilon, h, c) \rangle$                                                 |                                                                                                                                                                           |
| 742 | $\text{Red}^*(\varepsilon, c) \doteq c$                                                                                                                 | $\text{Fixpoint} : (C \times \mathcal{A}) \rightarrow \mathcal{A}$                                                                                                        |
| 743 | $\text{Red}^*(\kappa; \overline{\kappa}, c) \doteq \text{Red}^*(\overline{\kappa}, \text{Red}(\kappa, \bigwedge \theta(\kappa), c))$                    | $\text{Fixpoint}(c, \theta) \doteq \text{if } \exists \rho. \theta' \doteq \text{Weaken}(c, \theta, \rho) \text{ then } \text{Fixpoint}(c, \theta') \text{ else } \theta$ |
| 744 | $\text{Cert}_\theta : (Ctx \times \text{Proof} \times C) \rightarrow \text{Proof}$                                                                      |                                                                                                                                                                           |
| 745 | $\text{Cert}_\theta(\Gamma, q, \forall x : b. c) \doteq \lambda x. \text{Cert}_\theta(\Gamma; x : b, q \ x, c)$                                         | $\text{Weaken} : (C \times \mathcal{A} \times \text{Prefix}) \rightarrow \mathcal{A} + \perp$                                                                             |
| 746 | $\text{Cert}_\theta(\Gamma, q, p \rightarrow c) \doteq \lambda h_p. \text{Cert}_\theta(\Gamma; h_p : p, q \ h_p, c)$                                    | $\text{Weaken}(c, \theta, \rho) \doteq \text{if } I \subset \theta(\kappa) \text{ then } \theta[\kappa \mapsto I] \text{ else } \perp$                                    |
| 747 | $\text{Cert}_\theta(\Gamma, q, c_1 \wedge c_2) \doteq \langle \text{Cert}_\theta(\Gamma; L, q.1, c_1), \text{Cert}_\theta(\Gamma; R, q.2, c_2) \rangle$ | <b>where</b>                                                                                                                                                              |
| 748 | $\text{Cert}_\theta(\Gamma, q, \kappa(\bar{t})) \doteq \text{Oracle}(\Gamma[\theta] \vdash (\bigwedge \theta(\kappa))(\bar{t}))$                        | $\Gamma \vdash \kappa(\bar{t}) \doteq \text{Goal}(c, \rho)$                                                                                                               |
| 749 | $\text{Cert}_\theta(\Gamma, q, p) \doteq q$                                                                                                             | $I \doteq \{q_i \mid q_i \in \theta(\kappa), \text{Oracle}(\Gamma[\theta] \vdash q_i(\bar{t}))\}$                                                                         |
| 750 |                                                                                                                                                         |                                                                                                                                                                           |
| 751 |                                                                                                                                                         |                                                                                                                                                                           |

Fig. 13. The Fix procedure for reducing cyclic horn variables via predicate abstraction.

**Example** Consider a Horn variable  $\kappa_{\text{ex}} : \text{Int} \rightarrow \text{Int} \rightarrow \text{Prop}$  together with the binary qualifier

$$q_{\leq} \doteq \lambda z_0 : \text{Int}. \lambda z_1 : \text{Int}. z_0 \leq z_1$$

Each of its two parameters of sort *Int* can be mapped to either of the parameters of  $\kappa_{\text{ex}}$ . Hence, there are four  $\kappa_{\text{ex}}/q_{\leq}$  instances  $i_1 \doteq \{0 \mapsto 0, 1 \mapsto 0\}$ ,  $i_2 \doteq \{0 \mapsto 0, 1 \mapsto 1\}$ ,  $i_3 \doteq \{0 \mapsto 1, 1 \mapsto 0\}$  and  $i_4 \doteq \{0 \mapsto 1, 1 \mapsto 1\}$ , yielding the candidate set containing the four candidates

$$(q_{\leq})_{i_1} \doteq \lambda z_0 \lambda z_1. z_0 \leq z_0 \quad (q_{\leq})_{i_2} \doteq \lambda z_0 \lambda z_1. z_0 \leq z_1 \quad (q_{\leq})_{i_3} \doteq \lambda z_0 \lambda z_1. z_1 \leq z_0 \quad (q_{\leq})_{i_4} \doteq \lambda z_0 \lambda z_1. z_1 \leq z_1$$

**Assignments, Application and Validity** An *assignment*  $\theta \in \mathcal{A}$  maps each  $\kappa$  to a candidate set. We *apply*  $\theta$  to a Horn application, written  $\kappa(\bar{t})[\theta]$ , by applying the pointwise conjunction of the candidate set  $\theta(\kappa)$ . We lift application to contexts  $\Gamma[\theta]$  by applying to each hypothesis in  $\Gamma$ .

$$\begin{array}{lll}
\kappa(\bar{t})[\theta] & \doteq & (\bigwedge \theta(\kappa))(\bar{t}) \quad \text{horn applications} \\
t[\theta] & \doteq & t \quad \text{other atoms} \\
h : p ; \Gamma[\theta] & \doteq & h : p[\theta] ; \Gamma[\theta] \quad \text{hypotheses} \\
x : b ; \Gamma[\theta] & \doteq & x : b ; \Gamma[\theta] \quad \text{binders} \\
L ; \Gamma[\theta] & \doteq & L ; \Gamma[\theta] \quad \text{marks (R alike)}
\end{array}$$

An assignment  $\theta$  is *valid* for a  $\kappa$ -goal  $\Gamma \vdash \kappa(\bar{t})$  if the ( $\kappa$ -free) goal  $\Gamma[\theta] \vdash \kappa(\bar{t})[\theta]$  is valid. We write  $\theta \models c$  when  $\theta$  is valid for every  $\kappa$ -goal of  $c$ .

**Top-level Algorithm (Fix):** Fig. 13 summarizes the top-level Fix algorithm, which takes as input a constraint  $c$  and a set of qualifiers  $Q$  and returns as output a pair  $(c', q)$  comprising a rewritten constraint  $c'$  where all the horn variables in  $c$  are removed, and a proof  $q$  that shows that the rewritten  $c'$  implies the original  $c$ . The procedure Fix is analogous to Zap: first, it invokes PredAbs on the constraint and qualifiers to obtain an assignment  $\theta$ , this time for *all* the Horn variables simultaneously, as they are cyclic, and hence depend upon each other. Next, it folds Red over  $c$  and all the horn variables, to reduce the head applications of  $\kappa$  with  $\top$  and the body applications with the computed candidate set  $\theta(\kappa)$ . Finally, it invokes  $\text{Cert}_\theta$  to compute the proof  $q$  that  $c' \rightarrow c$ .

**Predicate Abstraction (PredAbs):** The right column in Fig. 13 shows how PredAbs takes as input a set of horn variables  $\overline{\kappa}$ , a constraint  $c$ , and a set of qualifiers  $Q$ , to compute an assignment that

is valid for  $c$ . This assignment is computed in two steps. First, we use `Init` to compute an *initial assignment* that maps each  $\kappa$  to the full set of candidates obtained from all  $\kappa/q$  instances of each qualifier in  $Q$ . Second, we invoke `Fixpoint` to distill the full set of candidates down into those that can be proven valid for each head  $\kappa$ -application (which lets `Fix` subsequently reduce to  $\top$ ).

**Fixpoint Computation** (`Fixpoint`): The procedure `Fixpoint`, summarized in Fig. 13, takes as input the constraint  $c$  and a candidate assignment  $\theta$ , and iteratively invokes `Weaken` to whittle away from  $\theta$  the candidates that *cannot be proven* valid. The procedure `Weaken` takes as input the constraint  $c$ , head-prefix  $\rho$  and assignment  $\theta$ , and uses the Oracle to determine the *subset of candidates* of  $\theta(\kappa)$ , named  $I$ , that can indeed be proven valid at the head corresponding to  $\rho$ . If this subset is strictly smaller than  $\theta(\kappa)$ , i.e., some whittling occurred, then `Weaken` returns the assignment where  $\kappa$  is updated to  $I$ ; otherwise (if the subset is unchanged), `Weaken` returns  $\perp$ . Now `Fixpoint` checks if there is *any* head-prefix  $\rho$  under which the (current) assignment  $\theta$  gets weakened to  $\theta'$ . If so, `Fixpoint` recurses on  $\theta'$ . Otherwise,  $\theta$  is valid for  $c$  and hence, is returned as the assignment. `Fixpoint` terminates: the initial assignment is finite, and each successful `Weaken` strictly decreases the total number of candidates. Note that `Weaken`, and hence `Fix`, is parameterized by the procedure Oracle, which attempts to *prove* a goal, which lets the user plug in theory-specific oracles to synthesize solutions for CHCs over arbitrary LEAN propositions.

**Certification** (`Certθ`): Finally, `Fix` invokes `Certθ` to justify the rewriting done by `Redθ` (where head  $\kappa$ -applications are replaced with  $\top$ ), by coughing up a *Proof* that shows that validity of the reduced constraint  $c'$  *implies* that of the original  $c$ . This procedure is nearly identical to the one for `Zap` discussed in § 4.3, in how it traverses the structure of  $c$  replaying in the proof from  $c'$ . The only difference is at the head applications  $\kappa(\bar{t})$  where (instead of navigating the structure of the solution) we invoke the Oracle to prove the applied conjunction  $(\bigwedge \theta(\kappa))(\bar{t})$ .

**Correctness** `Fix` requires that on any goal, Oracle either returns a valid proof term, or fails:

$$\text{ORACLE-SOUNDNESS} \frac{\text{Oracle}(\Gamma \vdash \varphi) = q}{\Gamma \vdash q : \varphi} \quad \text{ORACLE-COMPLETENESS} \frac{\Gamma \vdash \varphi}{\text{Oracle}(\Gamma \vdash \varphi) \text{ succeeds}}$$

**Theorem 4.3** (Fix-Soundness). *If Oracle is sound and  $\text{Fix}(c, Q) = (c', q)$ , then  $\vdash q : c' \rightarrow c$ .*

**Relative Completeness** It would be trivially sound, but not terribly useful to replace each horn application with  $\top$ . Fortunately, `Fix`( $c, Q$ ) does better: it returns the *strongest* valid solution expressible as conjunctions of predicates from  $Q$ . Given two  $Q$ -assignments, we say  $\theta$  is *stronger* than  $\theta'$ , written  $\theta \preceq \theta'$ , if for every  $\kappa$ , we have  $\theta'(\kappa) \subseteq \theta(\kappa)$ .

**Theorem 4.4** (Fix-Completeness [62]). *If Oracle is complete and  $\theta \doteq \text{PredAbs}(\exists \bar{\kappa}. c, Q)$ , then  $\theta \preceq \theta'$  for every  $Q$ -assignment  $\theta'$  with  $\theta' \models c$ .*

## 5 Foundational Verifiers

A CHC-based program verifier works in two stages, both of which are traditionally trusted: a frontend *generator* that takes a program and generates a constraint whose validity implies the program's safety; a backend *solver* that then determines the validity of the constraint. The solver in §4 removes the backend from the trusted base by generating a kernel checkable proof of the constraint's validity. Next, we show there is no need to trust the frontend either, by mechanizing the constraint generation such that a proof from the solver suffices to verify the source program. To this end, we develop two verifiers. First, in § 5.1, we present a Floyd-Hoare style verifier [24, 35] for an imperative language IMP with mutable state, where the Horn variables let us synthesize *loop invariants* [7]. Second, we present functional calculus  $\lambda_{RK}$  with refinement types (§ 5.2), use them to develop a sound declarative type system (§ 5.3), and use that to implement an algorithmic generator

|                    |                                                                                                                                        |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Variables</b>   | $x : \text{Var}$ (alias for <i>String</i> )                                                                                            |
| <b>States</b>      | $s : \text{Var} \rightarrow \text{Int}$                                                                                                |
| <b>Expressions</b> | $e : \text{State} \rightarrow \text{Int}$                                                                                              |
| <b>Guards</b>      | $g : \text{State} \rightarrow \text{Bool}$                                                                                             |
| <b>Assertions</b>  | $P : \text{State} \rightarrow \text{Prop}$                                                                                             |
| <b>Commands</b>    | $c ::= \text{skip} \mid x := e \mid c_1; c_2 \mid \text{if } g \text{ then } c_1 \text{ else } c_2 \mid \text{while } g \text{ do } c$ |

Fig. 14. Syntax of IMP, shallowly embedded into LEAN

(§ 5.4) that uses Horn variables to infer unknown *refinements*. In each case, we can then discharge the generated CHCs using the tactics from §4 to obtain an end-to-end foundational verifier.

### 5.1 A Verifier for IMP

To limber up, we implement a CHC based Floyd-Hoare style verifier for IMP summarized in Fig. 14. We shallowly embed IMP programs so states  $s$  are functions  $\text{Var} \rightarrow \mathbb{Z}$ , expressions  $e$  and guards  $g$  are state-indexed, and assertions  $P, Q$  are predicates  $\text{State} \rightarrow \text{Prop}$ . A (Floyd-Hoare) *triple* comprising a pre-condition  $P$ , command  $c$  and post-condition  $Q$  is *valid*, written  $\models \{P\} c \{Q\}$ , if every terminating run of  $c$  from a  $P$ -state ends in a  $Q$ -state.

**Constraint Generator** Let  $V \doteq \{x_1, \dots, x_n\}$  be a set of  $n$  ordered variables that occur in commands. The generator  $\text{VC}(P, c, Q)$  takes as input a triple  $P, c, Q$  (where  $c$  uses variables from  $V$ ) and outputs a CHC. The implementation is the textbook *weakest precondition*-based VC-generation method [18], except in two places. First, we do not require explicitly provided invariants for loops: when the generator hits a while it introduces a Horn *variable*  $\kappa$  for the unknown invariant — a  $|V|$ -ary predicate — and *constrains* it to satisfy the usual initial, body and exit obligations:

$$\begin{aligned}
 \text{VC}(P, \text{while } g \text{ do } c, Q) &\doteq \exists \kappa : \text{Int}^{|V|} \rightarrow \text{Prop}. \\
 &\quad \forall s. P(s) \rightarrow \llbracket \kappa \rrbracket(s) \quad (\text{initial}) \\
 &\quad \wedge \text{VC}(\lambda s. \llbracket \kappa \rrbracket(s) \wedge g(s), c, \llbracket \kappa \rrbracket) \quad (\text{body}) \\
 &\quad \wedge \forall s. \llbracket \kappa \rrbracket(s) \rightarrow \neg g(s) \rightarrow Q(s) \quad (\text{exit}) \\
 \text{where } \llbracket \kappa \rrbracket &\doteq \lambda s. \kappa(s(x_1), \dots, s(x_n))
 \end{aligned}$$

The above constraints are not in the syntax from Fig. 8 as we are quantifying over states  $s$  and not base-sorted values. Fortunately, LEAN's `simp` tactic suffices to reduce them to our grammar.

**Soundness** We prove in LEAN (Theorem D.3) that whenever the CHC returned by  $\text{VC}(P, c, Q)$  is satisfiable, that the corresponding triple is valid.

**Theorem 5.1** (VC-Generation soundness). *If  $\text{VC}(P, c, Q)$  then  $\models \{P\} c \{Q\}$ .*

### 5.2 Syntax and Semantics of $\lambda_{RK}$

Fig. 15 summarizes the syntax of  $\lambda_{RK}$ , which extends the simply typed  $\lambda$ -calculus with refinement types [9, 38], in particular, with Horn variables  $\kappa$  that enable refinement inference via CHC solving.

**Expressions and Evaluation** The grammar follows a standard call-by-value language with arithmetic and boolean expressions. Let bindings are required because the type system enforces ANF (A-Normal Form [23]) to accommodate dependent function applications [38]. We give expressions a standard, substitution-based, big-step, call-by-value operational semantics, written  $e \Downarrow v$  ( $e$  evaluates to  $v$ ). Type annotations are erased at runtime—( $e : \tau$ ) evaluates as  $e$ —so refinements play no part in evaluation and serve only for static checking.

|                    |                                                                                                                                                                                      |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                    | $x, y, v \in \text{Var} \quad \kappa \in \text{Kvar} \quad z \in \text{Int}$                                                                                                         |
| <b>Base sort</b>   | $\text{Base} \ni b ::= \text{Int} \mid \text{Bool}$                                                                                                                                  |
| <b>Term</b>        | $t_b ::= x_b \mid z \mid \text{true} \mid \text{false} \mid t_{\text{Int}} + t_{\text{Int}}$                                                                                         |
| <b>Formula</b>     | $\varphi ::= \top \mid t_b =_b t_b \mid t_{\text{Int}} \leq t_{\text{Int}} \mid \neg \varphi \mid \varphi \wedge \varphi \mid \varphi \rightarrow \varphi \mid \forall x:b. \varphi$ |
| <b>Refinements</b> | $r ::= \varphi \mid \kappa(\overline{t_b})$                                                                                                                                          |
| <b>Type</b>        | $\tau ::= \{ v:b \mid r \} \mid x:\tau \rightarrow \tau$                                                                                                                             |
| <b>Expression</b>  | $e ::= v \mid x \mid \oplus(\overline{e}) \mid e \ e \mid \text{let } x = e \text{ in } e \mid \text{if } e \text{ then } e \text{ else } e \mid (e : \tau)$                         |
| <b>Values</b>      | $\text{Val} \ni v ::= z \mid \text{true} \mid \text{false} \mid \lambda x. e$                                                                                                        |
| <b>Type env.</b>   | $\Gamma ::= \cdot \mid \Gamma, x:\tau$                                                                                                                                               |

 Fig. 15. Syntax of  $\lambda_{RK}$ .

**Refinements** We build refinement types on top of a *separate* first-order language, to eschew the circularities in the meta-theory (where types would depend on expressions, which would themselves contain types). This first-order language is stratified into intrinsically typed *terms* ( $t_b$ ) and *formulas* ( $\varphi$ ). A refinement  $r$  is either a first-order formula ( $\varphi$ ) over integers and booleans, or a Horn application  $\kappa(\overline{t_b})$ —highlighted in gray in Fig. 15—representing existentially quantified predicates over their arguments  $\overline{t_b}$  that the solver must infer.

**Types** A type  $\tau$  is either a *refined base type*  $\{ v:b \mid r \}$  or a *dependent arrow*  $x:\tau \rightarrow \tau$ , which names its argument  $x$  so the codomain may mention it. A refined base type restricts its sort  $b \in \{\text{Int}, \text{Bool}\}$  to the values  $v$  satisfying the refinement  $r$  (which may be an unknown Horn application).

Next, we provide a semantics for refinements by *interpreting* them as LEAN propositions, which provides a foundation for declarative typing (§ 5.3) and constraint generation (§ 5.4).

**Interpreting Formulas** We interpret each base type  $b$  as a LEAN type, writing  $\llbracket b \rrbracket$ , where  $\llbracket \text{Int} \rrbracket$  (resp.  $\llbracket \text{Bool} \rrbracket$ ) denotes the LEAN integers *Int* (resp. booleans *Bool*). A term  $t_b$  denotes an element of  $\llbracket b \rrbracket$  under a closing substitution  $\gamma : \text{Var} \rightarrow \text{Val}$  that maps its free variables to values, written  $\llbracket t_b \rrbracket_\gamma$ . A formula denotes a LEAN proposition over values and we write  $\llbracket \varphi \rrbracket_\gamma$  for its interpretation as a LEAN *Prop*. The interpretation is the natural one mapping constructs to their LEAN counterparts.

**Interpreting Refinements** To give meaning to a refinement  $r$  we must also interpret Horn applications  $\kappa(\overline{t_b})$ . We do this by quantifying at the meta level over a *K-assignment* which fixes the meaning of each  $\kappa$  as a LEAN predicate. Crucially, this *K* will itself map the *syntactic* horn variables to existentially bound LEAN predicates that the solver will then instantiate. Concretely, a *K-assignment* has the following LEAN type  $\text{Kvar} \rightarrow \text{List}(\Sigma b : \text{Base}, \llbracket b \rrbracket) \rightarrow \text{Prop}$ . Given a *K-assignment*, a Horn application is interpreted by looking up the predicate *K* assigns to  $\kappa$  and applying it to the interpreted arguments:

$$\llbracket \kappa(\overline{t_b}) \rrbracket_\gamma^K \doteq K \ \kappa \ \overline{(b, \llbracket t_b \rrbracket_\gamma)}$$

**Interpreting Types** Finally, we interpret types as predicates on values. We write  $v \in \llbracket \tau \rrbracket_\gamma^K$  to mean that the value  $v$  inhabits the interpretation of  $\tau$ . A refined base type denotes the base values satisfying the (interpretation) of its refinement, and a dependent arrow denotes the lambdas that send every argument in the domain to a result in the codomain:

$$v \in \llbracket \{ v:b \mid r \} \rrbracket_\gamma^K \doteq v \in \llbracket b \rrbracket \ \wedge \ \llbracket r \rrbracket_{\gamma[v \mapsto v]}^K$$

$$v \in \llbracket x:\tau_1 \rightarrow \tau_2 \rrbracket_\gamma^K \doteq v = \lambda x. e \ \wedge \ \forall v_a. v_a \in \llbracket \tau_1 \rrbracket_\gamma^K \Rightarrow \exists v_r. e[v_a/x] \Downarrow v_r \ \wedge \ v_r \in \llbracket \tau_2 \rrbracket_{\gamma[x \mapsto v_a]}^K$$

### 5.3 Declarative Typing for $\lambda_{RK}$

We define a declarative typing judgment  $\Gamma \vdash_K e : \tau$  which is mostly unremarkable except for being parameterized by a  $K$ -assignment. We highlight a couple of aspects of the system:

**Typing Variables with Selfification** When typing variables we strengthen its refinement by giving its *selfified* type [59], which crucially enables path-sensitive “occurrence” typing [71].

$$\frac{(x:\tau) \in \Gamma}{\Gamma \vdash_K x : \text{self}(x, \tau)} \text{ T-Var} \quad \begin{array}{l} \text{self}(x, \{v:b \mid p\}) \doteq \{v:b \mid p \wedge v = x\} \\ \text{self}(x, y : s \rightarrow t) \doteq y : s \rightarrow t \end{array}$$

**Typing Applications** The typing judgment enforces ANF so that a function is always applied to a *variable*, which can be substituted into the dependent codomain without placing an arbitrary expression inside a refinement. (The alternative is to extend the language with *existential types* [41], which exchanges the restriction for more complex subtyping and metatheory.)

$$\frac{\Gamma \vdash_K e : x:\tau_1 \rightarrow \tau_2 \quad \Gamma \vdash_K y : \tau_1}{\Gamma \vdash_K e y : \tau_2[y/x]} \text{ T-App}$$

**Subtyping** Most rules in the declarative system are syntax directed; subtyping is the only place where refinements are actually compared, so it is the place where essentially all of the interesting checking happens. A subsumption rule lets an expression of type  $\tau_1$  be used at any supertype  $\tau_2$ , deferring to a subtyping judgment  $\Gamma \vdash_K \tau_1 <: \tau_2$ . For arrows it is the standard rule, contravariant in the domain and covariant in the codomain; for two refined base types, subtyping holds when the first refinement implies the second under the assumptions in the context:

$$\frac{\forall \gamma, \langle \Gamma \rangle_\gamma^K \rightarrow \llbracket r_1 \rrbracket_\gamma^K \rightarrow \llbracket r_2 \rrbracket_\gamma^K}{\Gamma \vdash_K \{v:b \mid r_1\} <: \{v:b \mid r_2\}} \text{ S-BASE} \quad \frac{\Gamma \vdash_K \tau'_1 <: \tau_1 \quad \Gamma, x:\tau'_1 \vdash_K \tau_2 <: \tau'_2}{\Gamma \vdash_K x:\tau_1 \rightarrow \tau_2 <: x:\tau'_1 \rightarrow \tau'_2} \text{ S-FUN}$$

Here  $\langle \Gamma \rangle_\gamma^K$  *extracts* the assumptions from  $\Gamma$  by conjoining the base refinements

$$\langle \cdot \rangle_\gamma^K \doteq \top \quad \langle \Gamma, x:\{v:b \mid r\} \rangle_\gamma^K \doteq \langle \Gamma \rangle_\gamma^K \wedge \llbracket r[x/v] \rrbracket_\gamma^K \quad \langle \Gamma, x:\tau_1 \rightarrow \tau_2 \rangle_\gamma^K \doteq \langle \Gamma \rangle_\gamma^K$$

**Soundness** We prove the declarative system sound: a well-typed program never gets stuck, evaluating to a value that inhabits the interpretation of its type.

**Theorem 5.2** (Type soundness). *If  $\vdash_K e : \tau$ , then there exists a value  $v$  such that  $e \Downarrow v$  and  $v \in \llbracket \tau \rrbracket_\emptyset^K$ .*

### 5.4 A Verifier for $\lambda_{RK}$

The declarative system assumes a  $K$ -assignment with the solutions for Horn variables that make a program safe. Now we describe a constraint generation algorithm that produces a CHC that can be discharged by our solver to find such  $K$ -assignment.

First, we define the syntax of constraints  $c$  as described in Fig. 8 instantiating atoms as formulas  $\phi$ . Next, we define constraint generation as a *bidirectional* typechecking procedure implemented by three judgments: *synthesis*  $\Gamma \vdash e \Rightarrow \tau \rightsquigarrow c$ , *checking*  $\Gamma \vdash e \Leftarrow \tau \rightsquigarrow c$ , and *subtyping*  $\Gamma \vdash s <: t \rightsquigarrow c$ . Like the declarative system, most rules are syntax directed and accumulate subconstraints by conjoining them. For example, function application mirrors **T-APP** producing a constraint for each subexpression and conjoining them:

$$\frac{\Gamma \vdash e \Rightarrow x:\tau_1 \rightarrow \tau_2 \rightsquigarrow c_1 \quad \Gamma \vdash y \Leftarrow \tau_1 \rightsquigarrow c_2}{\Gamma \vdash e y \Rightarrow \tau_2[y/x] \rightsquigarrow c_1 \wedge c_2} \text{ SYN-APP}$$

| Case study | FLUX LoC | #CHCs | Spec LoC | Proof LoC |
|------------|----------|-------|----------|-----------|
| Sorting    | 316      | 15    | 96       | 620       |
| RingBuffer | 257      | 10    | 75       | 501       |
| TickTock   | 15       | 5     | —        | 302       |
| HashTable  | 562      | 22    | 245      | 1361      |
| <b>All</b> | 1250     | 52    | 416      | 2784      |

Table 1. Case studies: lines of verified Rust (LoC), number of constraints (#CHCs), lines of manually written LEAN specs (Spec LoC), and lines of proof (Proof LoC).

The interesting case is again subtyping on base refinements, which produces a constraint requiring that all values of base type  $b$  satisfying  $r_1$  also satisfy  $r_2$ :

$$\frac{}{\Gamma \vdash \{v:b \mid r_1\} <: \{v:b \mid r_2\} \rightsquigarrow \forall v:b. r_1 \rightarrow r_2} \text{S-BASE}$$

**Soundness** The constraint generated by our procedure must imply the safety of the program. We formalize this guarantee by extending the interpretation of refinements to constraints as follows:

$$\begin{aligned} \llbracket \top \rrbracket_Y^K &\doteq \top & \llbracket \forall x:b. c' \rrbracket_Y^K &\doteq \forall v \in \llbracket b \rrbracket. \llbracket c' \rrbracket_{Y[x \mapsto v]}^K \\ \llbracket r_1 \rightarrow r_2 \rrbracket_Y^K &\doteq \llbracket r_1 \rrbracket_Y^K \rightarrow \llbracket r_2 \rrbracket_Y^K & \llbracket c_1 \wedge c_2 \rrbracket_Y^K &\doteq \llbracket c_1 \rrbracket_Y^K \wedge \llbracket c_2 \rrbracket_Y^K \end{aligned}$$

Then we prove two theorems. The first connects constraint generation to declarative typing.

**Theorem 5.3** (Constraint Generation). *If  $\cdot \vdash e \Leftarrow \tau \rightsquigarrow c$  and  $\llbracket c \rrbracket^K$  is satisfiable, then  $\cdot \vdash_K e : \tau$ .*

Second, composing constraint generation soundness with declarative type soundness (Theorem 5.2) yields our end-to-end guarantee: if a program passes constraint generation and the resulting constraints are satisfiable, then the program evaluates to a value in its type's interpretation:

**Theorem 5.4** (Verifier Soundness). *If  $\cdot \vdash e \Leftarrow \tau \rightsquigarrow c$  and  $\llbracket c \rrbracket^K$  then  $\exists v$  s.t.  $e \Downarrow v$  and  $v \in \llbracket \tau \rrbracket^K$ .*

## 6 Implementation and Evaluation

We implemented FLEX in about 2700 lines of LEAN, and extended FLUX with a backend that emits constraints as LEAN propositions in about 1800 lines of Rust. Next, we describe experiments, using a workstation running Ubuntu 24.04.4 LTS with an AMD Ryzen 7 8845HS processor (8 cores, 16 threads) and 27GB of RAM, to evaluate FLEX on three questions

- **RQ1:** How *expressive* is the system compared to SMT? (§6.1)
- **RQ2:** How *efficient* is Zap over search-based tactics in LEAN? (§6.2)
- **RQ3:** How *scalable*, *automated*, and *costly* is the LEAN backend? (§6.3)

### 6.1 RQ1: Expressiveness

Through a suite of case studies, summarized in Table 1, we demonstrate how FLEX expands the scope of provable specifications by providing FLUX access to a foundational logic. The studies include Sorting: we verify the functional correctness of in-place insertion sort, quicksort, and merge sort, proving that each returns a sorted permutation of the input vector; RingBuffer: we verify memory safety of a ring-buffer-backed deque adapted from the Tock embedded OS kernel [64], where we prove that the implementation never reads from uninitialized memory, and that the Rust code implements a queue interface; TickTock: we verify various facts about modular arithmetic and bit-vectors needed to verify that the address segments calculated by the Tock embedded kernel implement process isolation [61]; HashTable: we verify the correctness of a hash table

```

1030 (a) The ring buffer (struct and impl)
1031   #[refined_by(cap:int, hd:int, tl:int,
1032               init: Map<int, bool>)]
1033   #[invariant(init_inv(self))]
1034   struct RingBuffer<'a, T: Copy + 'a> {
1035     buf: FSLc<'a, T>[cap, init],
1036     head: usize[hd],
1037     tail: usize[tl],
1038   }
1039   impl<'a, T: Copy> RingBuffer<'a, T> {
1040     ...
1041     #[proven_externally]
1042     fn dequeue(&mut self) -> Option<T> {
1043       if self.head != self.tail {
1044         let v = self.buf.get(self.head);
1045         self.head = (self.head+1) %
1046                   self.buf.len();
1047         Some(v)
1048       } else {
1049         None
1050       }
1051     }
1052   }
1053
1054 (b) Refinement-level specifications
1055   fn rb_len(rb: RingBuffer) -> int {
1056     if rb.tl > rb.hd { rb.tl - rb.hd }
1057     else if rb.tl < rb.hd { rb.cap - rb.hd + rb.tl }
1058     else { 0 }
1059   }
1060   fn valid_idx(rb: RingBuffer, idx: int) -> bool {
1061     (idx + rb.cap - rb.hd) % rb.cap < rb_len(rb)
1062   }
1063   // Defined in Lean
1064   fn init_inv(rb: RingBuffer) -> bool;
1065
1066 (c) Trusted wrapper over maybe uninitialized slices
1067   #[opaque]
1068   #[refined_by(len: int, init: Map<int, bool>)]
1069   struct FSLc<'a, T>(&'a mut [MaybeUninit<T>]);
1070
1071   impl<'a, T: Copy> FSLc<'a, T> {
1072     #[trusted]
1073     fn get(self: &FSLc<T>[@n, @f],
1074           idx: usize{idx < n && map_get(f, idx)}) -> T;
1075   }

```

Fig. 16. Ring buffer implementation adapted from Tock, verified to never read uninitialized memory. This requires a LEAN invariant, `init_inv`, that quantifies over valid indices and is not expressible in FLUX.

with chaining based collision resolution, by proving the Rust code implements a functional LEAN specification.

Previously, to ensure predictable verification, FLUX was restricted to quantifier- and recursion-free SMT-decidable specifications, which precluded expressing *any* of the case studies. For example, defining sortedness requires universally quantifying over the elements of an array, and the hash-table invariants quantify over bucket contents. Even when a property is expressible as it falls in the nominally decidable fragment — e.g. the bitvector facts from TickTock — the resulting formulas can make the SMT solver time out. This highlights a key benefit of FLEX: the ability to fall back to interactive proof when automation falls short.

**Example: RingBuffer Invariant** Fig. 16 shows snippets of the fixed-capacity `RingBuffer` implementation of a double-ended queue (deque) adapted from the Tock codebase, and simplified for exposition. The structure maintains `head` and `tail` indices that delimit the range of slots holding valid data. As elements are pushed and popped at either end, these indices move through the fixed-size buffer and may eventually *wrap around* (necessitating careful modular arithmetic reasoning.) Reading a slot outside the valid range is undefined behavior as it may contain uninitialized data. We track exactly which slots are initialized with a map `init` and maintain an invariant saying all valid indices are initialized. We can define the length (`rb_len`) and what indices are valid (`valid_idx`) as ordinary FLUX refinement functions that translate directly into LEAN definitions. The invariant `init_inv` requires quantifying over valid indices, which was not expressible as a FLUX refinement, but can now be, in LEAN:

```

def init_inv (rb : RingBuffer) : Prop :=
  ∀ i : Int, 0 ≤ i ∧ i < rb.cap → valid_idx rb i → rb.init i

```

| Suite           | #CHCs | Zap |    | Zap_grind |      |      | Zap_aesop |     |     |
|-----------------|-------|-----|----|-----------|------|------|-----------|-----|-----|
|                 |       | hb  | ms | #reduced  | ×hb  | ×ms  | #reduced  | ×hb | ×ms |
| flux-medium     | 78    | 268 | 24 | 48 (62%)  | 33×  | 23×  | 65 (83%)  | 99× | 62× |
| wave            | 35    | 518 | 49 | 20 (57%)  | 22×  | 16×  | 29 (83%)  | 41× | 25× |
| flux-tests      | 231   | 100 | 11 | 161 (70%) | 12×  | 9×   | 217 (94%) | 23× | 16× |
| liquid-fixpoint | 11    | 106 | 11 | 7 (64%)   | 14×  | 10×  | 10 (91%)  | 17× | 12× |
| flex-bench      | 14    | 585 | 62 | 11 (79%)  | 251× | 178× | 11 (79%)  | 87× | 44× |
| <b>All</b>      | 369   | 154 | 16 | 247 (67%) | 18×  | 13×  | 332 (90%) | 34× | 22× |

Table 2. RQ2: acyclic  $\kappa$ -variables reduction. #CHCs is the number of constraints in each suite; Zap can reduce all of them. For Zap, hb and ms are average heartbeats and wall-clock time (ms). For Zap\_grind and Zap\_aesop, #reduced is the number of constraints reduced, and ×hb/×ms report average cost relative to Zap.

Annotating the dequeue method with proven\_externally generates a LEAN constraint in which this definition is available to prove two key facts: that at the call to FS1c : get the precondition is satisfied, and that dequeue preserves init\_inv.

The LEAN backend does not eliminate all manual work. The Fix algorithm finds solutions automatically for simple cyclic  $\kappa$ -variables, but complex invariants (e.g., in merge and quicksort\_range) must still be supplied by hand. Sometimes adding the right qualifiers is sufficient; other times, even with the correct invariant provided, manual LEAN proof is needed to show it is maintained.

## 6.2 RQ2: Efficiency of Zap

Zap (§ 4.2) eliminates an acyclic  $\kappa$ -variable by instantiating the existential with the strongest solution for  $\kappa$ , and constructing a proof term that justifies replacing head occurrences of  $\kappa$  with  $\top$ . We demonstrate that deterministic proof construction makes Zap faster *and* robust than search-based tactics, and hence, crucial in practice.

**Experiment design.** To evaluate Zap we implemented two variants, Zap\_grind and Zap\_aesop, which use the same Sol procedure to compute the strongest solution, but replace the  $\text{Cert}_{\sigma,p}^{\kappa}$  proof-term construction procedure by calls to grind and aesop, respectively, each run with default settings. We ran each tactic on constraints with one or more acyclic  $\kappa$ -variables, to measure the cost of reduction using Zap (§ 4.2). Our benchmarks are divided into five suites: flux-tests: the full FLUX positive regression test suite; liquid-fixpoint: LIQUIDFIXPOINT’s regression tests, the SMT-based solver used by many refinement type systems, including FLUX; wave: a verified WebAssembly runtime previously ported to FLUX [39, 45]; flux-medium: medium-size benchmarks including implementations of vector-manipulating algorithms and programs from the FLUX guide and tutorials; flex-bench: the case studies from § 6.1.

**Results.** Table 2 summarizes the results across a total of 369 constraints. For each suite the table reports three metrics: the number of constraints successfully reduced and, over those, the average wall-clock time (ms) and heartbeats — LEAN’s deterministic proxy for proof effort. Zap can successfully reduce *all* constraints with a modest running time (max 62 ms). The search-based alternatives fare considerably worse. Zap\_grind reduces only 247 of the 369 constraints (67%), and on the constraints it can reduce, it consumes 18× more heartbeats and it is 13× slower on average; the performance is worst on flex-bench (11s per CHC), which has the largest constraints. Zap\_aesop fares better in coverage, successfully reducing 90% of the constraints, but it is 22 times slower on average than Zap. Furthermore, Proof terms produced by aesop are on average 10% deeper and 50% slower for the kernel to check, shown in Appendix E.

| Benchmark       | #CHCs | Success | #Cyclic | Success (cyclic) | LF time | FLEX Time |
|-----------------|-------|---------|---------|------------------|---------|-----------|
| flux-medium     | 148   | 89.2%   | 60      | 47 (78.3%)       | 4.2s    | 9.5m      |
| wave            | 77    | 96.1%   | 10      | 7 (70.0%)        | 2.8s    | 4.0m      |
| flux-tests      | 608   | 97.3%   | 80      | 67 (83.8%)       | 20.9s   | 7.4m      |
| liquid-fixpoint | 47    | 95.7%   | 14      | 14 (100.0%)      | 3.1s    | 58.0s     |
| <b>Total</b>    | 880   | 95.7%   | 164     | 135 (82.3%)      | 31s     | 21.8m     |

Table 3. Automation coverage of the full solver pipeline (Zap followed by Fix with a grind/aesop oracle), run with no human input. #CHCs is the number of constraints in the suite; Success is the fraction discharged, and Success (cyclic) the fraction discharged among those constraints with a cyclic  $\kappa$ -variable. LF time and FLEX Time are the total wall-clock time to solve the entire suite with Liquid Fixpoint (SMT) and FLEX, respectively.

### 6.3 RQ3: Scalability and Cost

SMT solvers are a marvelous feat of engineering. Replacing them with a *foundational* constraint solver in LEAN raises a couple of questions: does it *scale* to the constraints that verifiers generate in practice, and how much automation does it *trade off*? To answer these questions we gave FLEX free rein, running the full solver pipeline described in §4 — including Zap and Fix — on all of our benchmarks. For Fix, we instantiate the oracle that prunes instance candidates with a combination of grind and aesop. This leaves a  $\kappa$ -free VC, whose remaining goals we then attempt to close with the same grind/aesop combination. We raise LEAN’s default heartbeat, from 200,000 to 5,000,000, so that the search-based tactics have ample room to succeed. We exclude flex-bench from this experiment, as its case studies were chosen precisely because they require reasoning beyond what an SMT solver can reasonably automate.

**Results.** FLEX automatically discharges 95.7% of all constraints across the benchmark suite (Table 3). Failures concentrate on constraints with cyclic  $\kappa$ -variables, where the success rate drops to 82.3% due to addressable limitations in the current implementation of Fix. The 37 failures decompose into three categories. **(i) Recursion/Heartbeat limits:** 21 failures are caused by exceeding the default recursion depth limit during predicate abstraction, and a further 4 by exceeding the heartbeat limit; both parameters are tunable, and raising them can let additional constraints pass automatically at the cost of longer running times. **(ii) Missing qualifiers:** 4 failures occur because FLUX discovers some qualifiers by scraping the constraint, and we have not yet ported this inference to the LEAN backend; adding the missing qualifier manually allows our tactic to close these constraints. Even when all needed qualifiers are present, predicate abstraction can still fail to find the correct invariant, depending on the oracle it relies on. **(iii) Concrete Goal closure:** 8 failures have no  $\kappa$ -variables at all, and simply come down to grind/aesop being unable to close the goal automatically; adding annotations to relevant theorems can enable FLEX to handle more complex goals. This failure mode can also arise when Fix finds the correct invariant, as the verification condition remaining after  $\kappa$ -elimination must still be discharged by grind/aesop.

This demonstrates that our solver together with standard tactics can handle a variety of constraints automatically. Crucially, failures are *recoverable*: a user can fall back to an interactive LEAN proof rather than being blocked with only an opaque SMT failure to go on. Moreover, improvements to proof automation in LEAN would automatically transfer.

The expected cost is speed: Table 3 shows that the LEAN backend is roughly 2 orders of magnitude slower than the LIQUIDFIXPOINT SMT-based backend FLUX currently uses. For a foundational system whose proofs are machine-checked, this is the current price to pay. We expect that incorporating known optimizations from LIQUIDFIXPOINT into FLEX will shrink the performance gap.

## 7 Related Work

FLEX relates to a vast literature on program-verification; these are the lines closest to ours.

**SMT-based Verifiers** Our work is inspired by program logic [24, 35] based verifiers, in particular, those that emit a *constraint* that must be then proven to verify the program [18]. Typically these constraints are (horn variable free) VCs that are discharged by SMT solvers [15, 22, 48, 50]. PRUSTI [2] and VERUS [44] implement such Floyd-Hoare style verifiers for Rust. F\* generalizes the approach to the higher order setting using Dijkstra monads [67], while STAINLESS [30] uses refinement types. In contrast to the above, LIQUIDHASKELL [73], FLUX [45] and THRUST [58] use CHC, which generalize VCs with existentially quantified variables that delegate the synthesis of refinements or invariants to an SMT-based CHC solver. However, instead of relying upon incomplete, SMT heuristics [49], that can be opaque and hard to use [78], FLEX provides the full arsenal of interactive proof, when required, and provides foundational guarantees about the satisfiability of the CHC.

**Proving or Certifying VCs** Work on proof-carrying code showed how to avoid trusting SMT solvers by modifying decision procedures to emit certificates [54]. This idea was extended to model checkers [32], and refinement types [13], and several modern theorem provers emit independently checkable certificates [3, 4, 8, 19, 52]. A dual approach is to use foundational, interactive proof assistants to discharge VCs. For example, JAHOB [77] delegates VCs to a variety of backends including ISABELLE. Similarly the CREUSOT Rust verifier [16] emits VCs in the WHY3 format, after which they can be discharged using either SMT or interactive proof [20]. The above techniques apply to plain VCs: our work shows how to represent such provers as *oracles* and thus, lift such proofs to obtain certificates for CHC satisfiability.

**Transpiling to/from Interactive Provers** The above techniques use verification obligations as the medium of communication between implementation (e.g. C, Java, Rust, Haskell) and verification (e.g. SMT or interactive proof). An alternative is to *transpile* the implementation *into* the prover's language, and then prove properties about the transpiled code, as done by [37] and [65] which translate Scala and Haskell into ISABELLE or ROCQ respectively, or [5, 34] which translate Rust into LEAN. In general, though, the impedance mismatch between the languages can make such translation challenging. Of course, one can also go in the opposite direction, *i.e.* to *extract* the executable implementation from source written directly in a proof assistant, as done in systems like [1, 63] for C, or [25] for Rust or [27] for Dafny, which deeply embed program logic and refinement type based verifiers for the corresponding languages inside ROCQ or LEAN, after which the prover's extraction mechanism can be used to get executable code. FLEX is complementary to the above, in that we provide a foundational CHC solver, that can be used either to build end-to-end verifiers (§5) or to verify expressive constraints generated by (trusted) compiler plugins like FLUX (§6). Indeed, it would be interesting to explore using FLEX to build a CHC-based verifier on top of LEAN's MCVGen monad or the LOOM system.

**CHC Solvers** FLEX uses [14] to solve acyclic variables, and predicate abstraction [21] to compute (overapproximate) solutions for cyclic ones. In contrast, Spacer [42, 72], Eldarica [36], and SeaHorn [29] solve CHCs by inferring invariants using IC3 [10] or Lazy Abstraction and Craig Interpolation [33, 51], which do not require qualifiers, but which may diverge. It would be interesting to explore ways to extend FLEX with iterative refinement techniques to reduce the reliance on qualifiers.

**Mechanizing Refinement Types** Our mechanization of  $\lambda_{RK}$  is influenced by earlier mechanizations including simple refinement types [47], the work on System FR [30], Refined Featherweight Java [66] and most closely, the polymorphic refinement calculus of [9]. Unlike the prior work, we use a big-step semantics, and more importantly, mechanize refinement *inference* using Horn variables, algorithmic constraint generation and solving.

## References

- [1] Andrew W. Appel, Lennart Beringer, Robert Dockins, Aquinas Hobor, Xavier Leroy, and Gordon Stewart. 2014. *Program Logics for Certified Compilers*. Cambridge University Press, New York, NY, USA.
- [2] Vytautas Astrauskas, Aurea Bílá, Jonáš Fiala, Zachary Grannan, Christoph Matheja, Peter Müller, Federico Poli, and Alexander J. Summers. 2022. The Prusti Project: Formal Verification for Rust. In *NASA Formal Methods: 14th International Symposium, NFM 2022, Pasadena, CA, USA, May 24–27, 2022, Proceedings* (Pasadena, CA, USA). Springer-Verlag, Berlin, Heidelberg, 88–108. doi:10.1007/978-3-031-06773-0\_5
- [3] Haniel Barbosa, Clark Barrett, Byron Cook, Bruno Dutertre, Gereon Kremer, Hanna Lachnitt, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Cesare Tinelli, and Yoni Zohar. 2023. Generating and Exploiting Automated Reasoning Proof Certificates. *Commun. ACM* 66, 10 (Sept. 2023), 86–95. doi:10.1145/3587692
- [4] Filip Bártěk, Ahmed Bhayat, Robin Coutelier, Márton Hajdu, Matthias Hetzenberger, Petra Hozzová, Laura Kovács, Jakob Rath, Michael Rawson, Giles Reger, Martin Suda, Johannes Schoisswohl, and Andrei Voronkov. 2025. The Vampire Diary. arXiv:2506.03030 [cs.LO] <https://arxiv.org/abs/2506.03030>
- [5] Karthikeyan Bhargavan, Maxime Buyse, Lucas Franceschino, Lasse Letager Hansen, Franziskus Kiefer, Jonas Schneider-Bensch, and Bas Spitters. 2024. hax: Verifying Security-Critical Rust Software Using Multiple Provers. In *Verified Software. Theories, Tools and Experiments: 16th International Conference, VSTTE 2024, Prague, Czech Republic, October 14–15, 2024, Revised Selected Papers* (Prague, Czech Republic). Springer-Verlag, Berlin, Heidelberg, 96–119. doi:10.1007/978-3-031-86695-1\_7
- [6] Nikolaj Bjørner, Arie Gurfinkel, Ken McMillan, and Andrey Rybalchenko. 2015. Horn Clause Solvers for Program Verification. In *Fields of Logic and Computation II*. Springer, 24–51. doi:10.1007/978-3-319-23534-9\_2
- [7] Nikolaj S. Bjørner, Kenneth L. McMillan, and Andrey Rybalchenko. 2013. On Solving Universally Quantified Horn Clauses. In *Static Analysis - 20th International Symposium, SAS 2013, Seattle, WA, USA, June 20–22, 2013. Proceedings (Lecture Notes in Computer Science, Vol. 7935)*. Springer, 105–125. doi:10.1007/978-3-642-38856-9\_8
- [8] Jonas Bodingbauer, Márton Hajdu, Laura Kovács, Axel Polaczek, and Michael Rawson. 2026. Lean on Vampire Proofs (Short Paper). arXiv:2603.26342 [cs.LO] <https://arxiv.org/abs/2603.26342>
- [9] Michael H. Borkowski, Niki Vazou, and Ranjit Jhala. 2024. Mechanizing Refinement Types. *Proc. ACM Program. Lang.* 8, POPL, Article 70 (Jan. 2024), 30 pages. doi:10.1145/3632912
- [10] Aaron R Bradley. 2011. SAT-based model checking without unrolling. In *Verification, Model Checking, and Abstract Interpretation (VMCAI)*. Springer, 70–87. doi:10.1007/978-3-642-18275-4\_7
- [11] Mauro Bringolf, Dominik Winterer, and Zhendong Su. 2023. Finding and Understanding Incompleteness Bugs in SMT Solvers. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering* (Rochester, MI, USA) (ASE '22). Association for Computing Machinery, New York, NY, USA, Article 43, 10 pages. doi:10.1145/3551349.3560435
- [12] Mario Carneiro. 2025. Lean4Lean: Verifying a Typechecker for Lean, in Lean. arXiv:2403.14064 [cs.PL] <https://arxiv.org/abs/2403.14064>
- [13] Juan Chen, Ravi Chugh, and Nikhil Swamy. 2010. Type-preserving compilation of end-to-end verification of security enforcement. In *Proceedings of the 2010 ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2010, Toronto, Ontario, Canada, June 5–10, 2010*, Benjamin G. Zorn and Alex Aiken (Eds.). ACM, 412–423. doi:10.1145/1806596.1806643
- [14] Benjamin Cosman and Ranjit Jhala. 2017. Local refinement typing. *Proc. ACM Program. Lang.* 1, ICFP, Article 26 (Aug. 2017), 27 pages. doi:10.1145/3110270
- [15] Pascal Cuoq, Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. 2012. Frama-C: a software analysis perspective. In *Proceedings of the 10th International Conference on Software Engineering and Formal Methods* (Thessaloniki, Greece) (SEFM'12). Springer-Verlag, Berlin, Heidelberg, 233–247. doi:10.1007/978-3-642-33826-7\_16
- [16] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. 2022. Creusot: A Foundry for the Deductive Verification of Rust Programs. In *Formal Methods and Software Engineering: 23rd International Conference on Formal Engineering Methods, ICFEM 2022, Madrid, Spain, October 24–27, 2022, Proceedings* (Madrid, Spain). Springer-Verlag, Berlin, Heidelberg, 90–105. doi:10.1007/978-3-031-17244-1\_6
- [17] David Detlefs, Greg Nelson, and James B. Saxe. 2005. Simplify: A Theorem Prover for Program Checking. *J. ACM* 52, 3 (2005), 365–473. doi:10.1145/1066100.1066102
- [18] Edsger W. Dijkstra. 1976. *A Discipline of Programming*. Prentice-Hall, Englewood Cliffs, N.J.
- [19] Burak Ekici, Alain Mebsout, Cesare Tinelli, Chantal Keller, Guy Katz, Andrew Reynolds, and Clark Barrett. 2017. SMTCoq: A plug-in for integrating SMT solvers into Coq. In *Computer Aided Verification - 29th International Conference*. Heidelberg, Germany. <https://hal.science/hal-01669345>
- [20] Jean-Christophe Filliâtre and Andrei Paskevich. 2013. Why3: where programs meet provers. In *Proceedings of the 22nd European Conference on Programming Languages and Systems* (Rome, Italy) (ESOP'13). Springer-Verlag, Berlin,

- Heidelberg, 125–128. doi:10.1007/978-3-642-37036-6\_8
- [21] Cormac Flanagan and K. Rustan M. Leino. 2001. Houdini, an Annotation Assistant for ESC/Java. In *FME 2001: Formal Methods for Increasing Software Productivity, International Symposium of Formal Methods Europe, Berlin, Germany, March 12-16, 2001, Proceedings (Lecture Notes in Computer Science, Vol. 2021)*, José Nuno Oliveira and Pamela Zave (Eds.). Springer, 500–517. doi:10.1007/3-540-45251-6\_29
- [22] Cormac Flanagan, K. Rustan M. Leino, Mark Lillibridge, Greg Nelson, James B. Saxe, and Raymie Stata. 2002. Extended static checking for Java. In *Proceedings of the ACM SIGPLAN 2002 Conference on Programming Language Design and Implementation* (Berlin, Germany) (*PLDI '02*). Association for Computing Machinery, New York, NY, USA, 234–245. doi:10.1145/512529.512558
- [23] Cormac Flanagan, Amr Sabry, Bruce F. Duba, and Matthias Felleisen. 1993. The essence of compiling with continuations. *SIGPLAN Not.* 28, 6 (June 1993), 237–247. doi:10.1145/173262.155113
- [24] Robert W. Floyd. 1967. Assigning Meanings to Programs. In *Mathematical Aspects of Computer Science*, J. T. Schwartz (Ed.), Vol. 19. American Mathematical Society, 19–32.
- [25] Lennard Gäher, Michael Sammler, Ralf Jung, Robbert Krebbers, and Derek Dreyer. 2024. RefinedRust: A Type System for High-Assurance Verification of Rust Programs. *Proc. ACM Program. Lang.* 8, PLDI, Article 192 (June 2024), 25 pages. doi:10.1145/3656422
- [26] Catarina Gamboa, Abigail Reese, Alcides Fonseca, and Jonathan Aldrich. 2025. Usability Barriers for Liquid Types. *Proc. ACM Program. Lang.* 9, PLDI, Article 224 (June 2025), 26 pages. doi:10.1145/3729327
- [27] Vladimir Gladshtein, George Pirlea, Qiyan Zhao, Vitaly Kurin, and Ilya Sergey. 2026. Foundational Multi-Modal Program Verifiers. *Proc. ACM Program. Lang.* 10, POPL, Article 77 (Jan. 2026), 32 pages. doi:10.1145/3776719
- [28] Susanne Graf and Hassen Saidi. 1997. Construction of abstract state graphs with PVS. In *Computer Aided Verification*, Orna Grumberg (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 72–83.
- [29] Arie Gurfinkel, Temesghen Kahsai, Anvesh Komuravelli, and Jorge A. Navas. 2015. The SeaHorn Verification Framework. In *International Conference on Computer Aided Verification*. 343–361. doi:10.1007/978-3-319-21690-4\_20
- [30] Jad Hamza, Nicolas Voirol, and Viktor Kunčák. 2019. System FR: formalized foundations for the stainless verifier. *Proc. ACM Program. Lang.* 3, OOPSLA, Article 166 (Oct. 2019), 30 pages. doi:10.1145/3360592
- [31] Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jay R. Lorch, Bryan Parno, Justine Stephenson, Srinath Setty, and Brian Zill. 2015. IronFleet: Proving Practical Distributed Systems Correct. In *Proceedings of the 25th Symposium on Operating Systems Principles* (Monterey, California, USA) (*SOSP '15*). Association for Computing Machinery, New York, NY, USA, 1–16. doi:10.1145/2815400.2815428
- [32] Thomas A. Henzinger, Ranjit Jhala, Rupak Majumdar, George C. Necula, Grégoire Sutre, and Westley Weimer. 2002. Temporal-Safety Proofs for Systems Code. In *Proceedings of the 14th International Conference on Computer Aided Verification (CAV '02)*. Springer-Verlag, Berlin, Heidelberg, 526–538.
- [33] Thomas A. Henzinger, Ranjit Jhala, Rupak Majumdar, and Grégoire Sutre. 2002. Lazy abstraction. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Portland, Oregon) (*POPL '02*). Association for Computing Machinery, New York, NY, USA, 58–70. doi:10.1145/503272.503279
- [34] Son Ho and Jonathan Protzenko. 2022. Aeneas: Rust verification by functional translation. *Proc. ACM Program. Lang.* 6, ICFP, Article 116 (Aug. 2022), 31 pages. doi:10.1145/3547647
- [35] Charles Anthony Richard Hoare. 1969. An axiomatic basis for computer programming. *Commun. ACM* 12, 10 (1969), 576–580.
- [36] Hossein Hojjat and Philipp Rümmer. 2018. The ELDARICA Horn Solver. In *2018 Formal Methods in Computer Aided Design (FMCAD)*. 1–7. doi:10.23919/FMCAD.2018.8603013
- [37] Lars Hupel and Viktor Kuncak. 2016. Translating Scala Programs to Isabelle/HOL. In *Proceedings of the 8th International Joint Conference on Automated Reasoning - Volume 9706*. Springer-Verlag, Berlin, Heidelberg, 568–577. doi:10.1007/978-3-319-40229-1\_38
- [38] Ranjit Jhala and Niki Vazou. 2021. Refinement Types: A Tutorial. *Found. Trends Program. Lang.* 6, 3–4 (Oct. 2021), 159–317. doi:10.1561/25000000032
- [39] Evan Johnson, Evan Laufer, Zijie Zhao, Dan Gohman, Shravan Narayan, Stefan Savage, Deian Stefan, and Fraser Brown. 2023. WaVe: a verifiably secure WebAssembly sandboxing runtime. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2940–2955.
- [40] Richard M. Karp. 1972. Reducibility Among Combinatorial Problems. In *Proceedings of a symposium on the Complexity of Computer Computations, held March 20-22, 1972, at the IBM Thomas J. Watson Research Center, Yorktown Heights, New York, USA (The IBM Research Symposia Series)*, Raymond E. Miller and James W. Thatcher (Eds.). Plenum Press, New York, 85–103. doi:10.1007/978-1-4684-2001-2\_9
- [41] Kenneth Knowles and Cormac Flanagan. 2009. Compositional reasoning and decidable checking for dependent contract types. In *Proceedings of the 3rd Workshop on Programming Languages Meets Program Verification* (Savannah, GA, USA) (*PLPV '09*). Association for Computing Machinery, New York, NY, USA, 27–38. doi:10.1145/1481848.1481853

- [42] Anvesh Komuravelli, Arie Gurfinkel, and Sagar Chaki. 2016. SMT-based model checking for recursive programs. *Form. Methods Syst. Des.* 48, 3 (June 2016), 175–205. doi:10.1007/s10703-016-0249-4
- [43] Andrea Lattuada, Travis Hance, Jay Bosamiya, Matthias Brun, Chanhee Cho, Hayley LeBlanc, Pranav Srinivasan, Reto Achermann, Tej Chajed, Chris Hawblitzel, Jon Howell, Jacob R. Lorch, Oded Padon, and Bryan Parno. 2024. Verus: A Practical Foundation for Systems Verification. In *Proceedings of the 30th ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*. 438–454. doi:10.1145/3694715.3695952
- [44] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2023. Verus: Verifying Rust Programs using Linear Ghost Types. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 85 (April 2023), 30 pages. doi:10.1145/3586037
- [45] Nico Lehmann, Adam T. Geller, Niki Vazou, and Ranjit Jhala. 2023. Flux: Liquid Types for Rust. *Proc. ACM Program. Lang.* 7, PLDI, Article 169 (June 2023), 25 pages. doi:10.1145/3591283
- [46] Nico Lehmann, Rose Kunkel, Jordan Brown, Jean Yang, Niki Vazou, Nadia Polikarpova, Deian Stefan, and Ranjit Jhala. 2021. STORM: Refinement Types for Secure Web Applications. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 441–459. <https://www.usenix.org/conference/osdi21/presentation/lehmann>
- [47] Nico Lehmann and Éric Tanter. 2016. Formalizing Simple Refinement Types in Coq. In *2nd International Workshop on Coq for Programming Languages (CoqPL '16)*. St. Petersburg, FL, USA.
- [48] K. Rustan M. Leino. 2010. Dafny: an automatic program verifier for functional correctness. In *Proceedings of the 16th International Conference on Logic for Programming, Artificial Intelligence, and Reasoning (Dakar, Senegal) (LPAR'10)*. Springer-Verlag, Berlin, Heidelberg, 348–370.
- [49] K. Rustan M. Leino and Clément Pit-Claudel. 2016. Trigger Selection Strategies to Stabilize Program Verifiers. In *Computer Aided Verification - 28th International Conference, CAV 2016, Toronto, ON, Canada, July 17-23, 2016, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 9779)*. Springer, 361–381. doi:10.1007/978-3-319-41528-4\_20
- [50] D. C. Luckham, S. M. German, F. W. von Henke, R. A. Karp, P. W. Milne, D. C. Oppen, W. Polak, and W. L. Scherlis. 1979. *Stanford Pascal Verifier user manual*. Technical Report STAN-CS-79-731. Stanford University, Department of Computer Science.
- [51] Kenneth L. McMillan. 2006. Lazy abstraction with interpolants. In *International Conference on Computer Aided Verification (CAV)*. Springer, 123–136. doi:10.1007/11817963\_14
- [52] Abdalrhman Mohamed, Tomaz Mascarenhas, Harun Khan, Haniel Barbosa, Andrew Reynolds, Yicheng Qian, Cesare Tinelli, and Clark Barrett. 2025. lean-smt: An SMT Tactic for Discharging Proof Goals in Lean. In *Computer Aided Verification*, Ruzica Piskac and Zvonimir Rakamarić (Eds.). Springer Nature Switzerland, Cham, 197–212.
- [53] Eric Mugnier, Yuanyuan Zhou, Ranjit Jhala, and Michael Coblenz. 2025. On the Impact of Formal Verification on Software Development. *Proc. ACM Program. Lang.* 9, OOPSLA2, Article 403 (Oct. 2025), 27 pages. doi:10.1145/3763181
- [54] George Ciprian Necula. 1998. *Compiling with Proofs*. Ph. D. Dissertation. Carnegie Mellon University, Pittsburgh, PA, USA. <https://www.cs.cmu.edu/~rwh/students/necula.pdf> Available as Technical Report CMU-CS-98-154.
- [55] Charles Gregory Nelson. 1980. *Techniques for Program Verification*. Ph. D. Dissertation. Stanford University, Stanford, CA, USA.
- [56] Tobias Nipkow and Gerwin Klein. 2014. *Concrete Semantics with Isabelle/HOL*. Springer. <http://concrete-semantics.org>
- [57] Ulf Norell. 2009. *Dependently Typed Programming in Agda*. Springer Berlin Heidelberg, Berlin, Heidelberg, 230–266. doi:10.1007/978-3-642-04652-0\_5
- [58] Hiromi Ogawa, Taro Sekiyama, and Hiroshi Unno. 2025. Thrust: A Prophecy-Based Refinement Type System for Rust. *Proc. ACM Program. Lang.* 9, PLDI, Article 230 (June 2025), 25 pages. doi:10.1145/3729333
- [59] Xinming Ou, Gang Tan, Yitzhak Mandelbaum, and David Walker. 2004. Dynamic Typing with Dependent Types. In *Exploring New Frontiers of Theoretical Informatics*, Jean-Jacques Levy, Ernst W. Mayr, and John C. Mitchell (Eds.). Springer US, Boston, MA, 437–450.
- [60] Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hrițcu, Vilhelm Sjöberg, and Brent Yorgey. 2026. *Logical Foundations*. Software Foundations, Vol. 1. Electronic textbook. <https://softwarefoundations.cis.upenn.edu/lf-current/>
- [61] Vivien Rindisbacher, Evan Johnson, Nico Lehmann, Tyler Potyondy, Pat Pannuto, Stefan Savage, Deian Stefan, and Ranjit Jhala. 2025. TickTock: Verified Isolation in a Production Embedded OS. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles (Lotte Hotel World, Seoul, Republic of Korea) (SOSP '25)*. Association for Computing Machinery, New York, NY, USA, 786–801. doi:10.1145/3731569.3764856
- [62] Patrick M. Rondon, Ming Kawaguchi, and Ranjit Jhala. 2008. Liquid types. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (Tucson, AZ, USA) (PLDI '08)*. Association for Computing Machinery, New York, NY, USA, 159–169. doi:10.1145/1375581.1375602
- [63] Michael Sammler, Rodolphe Lépigre, Robbert Krebbers, Kayvan Memarian, Derek Dreyer, and Deepak Garg. 2021. RefinedC: automating the foundational verification of C code with refined ownership types. In *Proceedings of the 42nd*

- ACM SIGPLAN International Conference on Programming Language Design and Implementation (Virtual, Canada) (PLDI 2021). Association for Computing Machinery, New York, NY, USA, 158–174. doi:10.1145/3453483.3454036
- [64] Leon Schuermann, Brad Campbell, Branden Ghena, Philip Levis, Amit Levy, and Pat Pannuto. 2025. Tock: From Research to Securing 10 Million Computers. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*. 36–49.
- [65] Antal Spector-Zabusky, Joachim Breitner, Christine Rizkallah, and Stephanie Weirich. 2018. Total Haskell is Reasonable Coq. In *Proceedings of the 7th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2018)*. Association for Computing Machinery, 152–164. doi:10.1145/3167092
- [66] Ke Sun, Di Wang, Sheng Chen, Meng Wang, and Dan Hao. 2024. Formalizing, Mechanizing, and Verifying Class-Based Refinement Types. In *38th European Conference on Object-Oriented Programming (ECOOP 2024) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 313)*, Jonathan Aldrich and Guido Salvaneschi (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 39:1–39:30. doi:10.4230/LIPIcs.ECOOP.2024.39
- [67] Nikhil Swamy, Joel Weinberger, Cole Schlesinger, Juan Chen, and Benjamin Livshits. 2013. Verifying higher-order programs with the dijkstra monad. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Seattle, Washington, USA) (PLDI '13). Association for Computing Machinery, New York, NY, USA, 387–398. doi:10.1145/2491956.2491978
- [68] Robert Tarjan. 1972. Depth-First Search and Linear Graph Algorithms. *SIAM J. Comput.* 1, 2 (1972), 146–160. doi:10.1137/0201010
- [69] The Coq Development Team. 2024. The Coq Reference Manual – Release 8.19.0. <https://coq.inria.fr/doc/V8.19.0/refman>.
- [70] The mathlib Community. 2020. The Lean Mathematical Library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*. ACM, New Orleans, LA, USA. doi:10.1145/3372885.3373824
- [71] Sam Tobin-Hochstadt and Matthias Felleisen. 2008. The design and implementation of typed scheme. *SIGPLAN Not.* 43, 1 (Jan. 2008), 395–406. doi:10.1145/1328897.1328486
- [72] Takeshi Tsukada and Hiroshi Unno. 2024. Inductive Approach to Spacer. *Proc. ACM Program. Lang.* 8, PLDI, Article 227 (June 2024), 24 pages. doi:10.1145/3656457
- [73] Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. 2014. Refinement types for Haskell. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming* (Gothenburg, Sweden) (ICFP '14). Association for Computing Machinery, New York, NY, USA, 269–282. doi:10.1145/2628136.2628161
- [74] Dominik Winterer and Zhendong Su. 2024. Validating SMT Solvers for Correctness and Performance via Grammar-Based Enumeration. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 355 (Oct. 2024), 24 pages. doi:10.1145/3689795
- [75] Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2020. On the unusual effectiveness of type-aware operator mutations for testing SMT solvers. *Proc. ACM Program. Lang.* 4, OOPSLA, Article 193 (Nov. 2020), 25 pages. doi:10.1145/3428261
- [76] Dominik Winterer, Chengyu Zhang, and Zhendong Su. 2020. Validating SMT solvers via semantic fusion. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (London, UK) (PLDI 2020). Association for Computing Machinery, New York, NY, USA, 718–730. doi:10.1145/3385412.3385985
- [77] Karen Zee, Viktor Kuncak, and Martin Rinard. 2008. Full functional verification of linked data structures. In *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Tucson, AZ, USA) (PLDI '08). Association for Computing Machinery, New York, NY, USA, 349–361. doi:10.1145/1375581.1375624
- [78] Yi Zhou, Jay Bosamiya, Yoshiki Takashima, Jessica Li, Marijn Heule, and Bryan Parno. 2023. Mariposa: Measuring SMT Instability in Automated Program Verification. In *2023 Formal Methods in Computer-Aided Design (FMCAD)*. 178–188. doi:10.34727/2023/isbn.978-3-85448-060-0\_26
- [79] Jean-Karim Zinzindohoué, Karthikeyan Bhargavan, Jonathan Protzenko, and Benjamin Beurdouche. 2017. HACL\*: A Verified Modern Cryptographic Library. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 1789–1806.